abyss

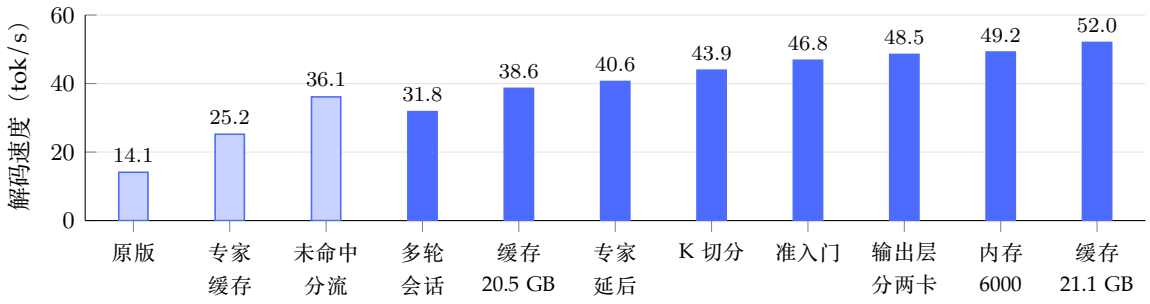
Abyss：在两张显卡上分层运行 DeepSeek-V4.1-Flash

Abyss

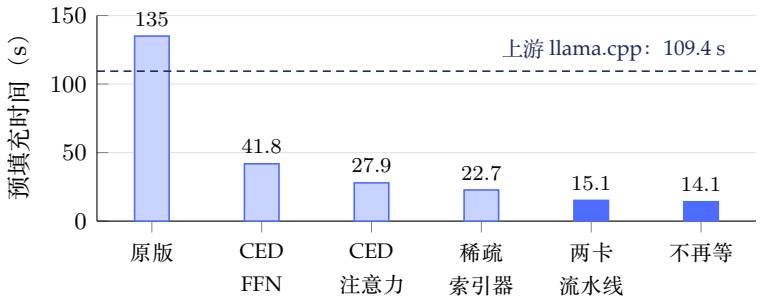
2026 年 11 月

摘要

长上下文智能体把推理的压力从算力推到了显存、内存和 PCIe 上。DeepSeek-V4.1-Flash 总参数 763B，原生权重约 474 GiB，超过一台双卡工作站（2 × RTX PRO 6000 Max-Q，显存合计 191 GiB；内存 192 GB）的全部容量。我们在 mx-llama.cpp 上实现了一个分层推理调度器 Abyss：编码器各层的专家常驻显存，解码器后 18 层的专家放在主机内存，由 GPU 上的专家缓存和 CPU 分担，Engram 大表留在 NVMe。在保持原生 MXFP4 精度的前提下（每项改动对改动前的差别都在数值底线以内），我们围绕这款模型的编码器-解码器结构（CED）、压缩注意力和稀疏索引器做了四十多个补丁。预填充方面，CED 感知预填充让解码器层只算后面还看得到的 token，加上官方稀疏索引器、Engram 预取、预填充大批次分流、末段一遍算完和长提示词的两卡流水线，4.8 万 token 提示词从 135 s 降到约 14 s（约 3,400 tok/s），多轮智能体之后每轮的预填充从 5.7–7.2 s 降到约 2 s。解码方面，GPU 专家缓存、按热度准入并交给 CPU 的未命中、专家延后（CPU 算的专家晚一层加回残差）、缓存的准入门、小内核融合和输出层两卡各算一半等，把单流解码从 14.1 tok/s 提到只追加的多轮会话里约 52 tok/s（9.6 万上下文；生产上按请求约 50 tok/s）；上下文从 12.8 万扩到 19.2 万，内存从 3600 提到 6000 MT/s。同一份权重、同样的启动参数下，上游 llama.cpp 的解码慢约 20%，长提示词的预填充慢 5 倍以上；用三个公开智能体 benchmark（MLPerf Agentic Inference、InferenceX AgentX、TraceLab）的数据和客户端跑的子集结论一致（都不是官方结果），同样的工作量我们用时约为上游的一半，只有 TraceLab 的首 token 时间中位数因为一处检查点的取舍输给上游。服务侧，我们按官方 `encoding.py` 重写了对话模板，按官方 Recipe 的规则编码请求文字里的控制 token，让上一轮的回答不再每轮重算，并修复了真实 Codex 和 Hermes 负载暴露的多处缺陷；权重换成了 DeepSeek 10 月 17 日更新的 1017 版（自己转换，PPL 低 3.4%）。我们同时记录了评估方法（两档数值底线、对官方实现的对拍、解码路径的 PPL、固定种子的测速）、两个让结论走偏的测法陷阱（同一提示词重复生成会让解码虚高，nsys 按节点跟踪会放大 CUDA graph 的启动），以及实现或测过、但没有采用的路：专家预取、DSpark 投机解码、每 token 少用一个专家、输出层 Q8_0 和第 0–19 层的张量并行。



(a)



(b)

图 1 | (a) 单流解码速度，依次叠加各项优化（第 4 节）。浅色的 3 根是 4.8 万上下文、同一提示词第二次生成，缓存命中偏高；深色的是 9.6 万上下文、只追加的多轮会话，各取那项改动自己那次对比里「改动后」的数（§4.4–§4.14、§7），「缓存 20.5 GB」含上传比例改为 0.3，「K 切分」含专家统计不再切图，「缓存 21.1 GB」含矩阵乘的尾部和大图分两段。同样的会话上上游 llama.cpp 是 39.3 tok/s（§11.2）。(b) 4.8 万 token 提示词的预填充时间（第 5 节）：浅色的 4 根 Engram 页缓存已热；深色的 2 根是智能体会话的首轮、页缓存清空（§5.7、§5.9）；虚线是上游 llama.cpp，页缓存清空。

目录

1 引言 **5**

2 硬件与模型 **6**

2.1 硬件 6

2.2 模型 6

2.3 权重格式与量化 10

2.4 方案 B 的布局 11

3 评估方法 **13**

3.1 数值底线 13

3.2 对官方实现的对拍 13

3.3 剖析 13

3.4 解码路径的 PPL 14

3.5 固定种子与冷页缓存 14

3.6 标准口径 14

4 解码 **15**
4.1 GPU 专家缓存 . . . 15
4.2 解码注意力只取 top-k 行 . . . 16
4.3 按热度准入的未命中分流 . . . 16
4.4 实际会话中的解码速度 . . . 17
4.5 腾出显存给专家缓存 . . . 18
4.6 10-28 的剖析 . . . 19
4.7 专家的预测与预取 . . . 19
4.8 专家延后 . . . 20
4.9 专家统计不再切图 . . . 21
4.10 GPU 上的零碎开销 . . . 21
4.11 解码时的 Engram 读盘 . . . 22
4.12 缓存的准入门与淘汰 . . . 23
4.13 采样与 token 之间 . . . 23
4.14 输出层分两卡 . . . 24
4.15 现在的时间分布 . . . 24
4.16 测过但没有采用的 . . . 25

5 预填充 **26**
5.1 CED 感知预填充 . . . 26
5.2 Engram 预取 . . . 27
5.3 补上官方稀疏索引器 . . . 27
5.4 大批次分流 . . . 28
5.5 末段一遍算完 . . . 29
5.6 大批次用缓存 . . . 29
5.7 长提示词的两卡流水线 . . . 30
5.8 检查点拆分出的小调用 . . . 32
5.9 大批次上传不再让主机等 . . . 32

6 服务与智能体负载 **33**
6.1 多轮会话的缓存复用 . . . 33
6.2 prompt 缓存的选择 . . . 33
6.3 真实负载暴露的缺陷 . . . 33
6.4 重启后的长对话首次预填充 . . . 34
6.5 对话模板与采样 . . . 35
6.6 请求文字里的控制 token . . . 35
6.7 槽位被只共享开头的请求占用 . . . 36
6.8 智能体客户端的额外请求 . . . 36
6.9 上一轮回答不再重算 . . . 36
6.10 CUDA graph 空闲后不清 . . . 37

6.11 多请求系统的前缀缓存 37

7 把内存提到 5200、6000 MT/s **38**

8 把上下文扩到 19.2 万 **39**

9 换用 1017 权重 **41**

10 第 0–19 层的张量并行原型 **42**

11 和上游 llama.cpp 对比 **43**

11.1 代码 43

11.2 同一份权重的实测 44

11.3 三个公开 benchmark 的子集 45

12 结论、局限与未来方向 **47**

A 补丁清单 **51**

B 复现 **52**

C 术语表 **55**

1 引言

智能体把大模型的典型请求从「短问短答」变成了「长上下文、多轮、带工具调用」。一个编程智能体每一轮都要在几万 token 的上下文后面追加回复、工具结果或追问，再让模型继续生成。这种负载有两个特点：一是上下文长，注意力和 KV 的开销随之增长；二是每一轮新增的内容不多，但首 token 之前的那段预填充决定了用户等待的时间。

DeepSeek-V4.1-Flash [1] 是为这类负载设计的。它有 552B 主干参数、196.6B 的 Engram 查表和约 14B 的 DSpark 草稿模型；40 层中前 20 层是因果编码器、后 20 层是解码器（CED），提示词的每个 token 只需走完编码器，生成的 token 才走完全部 40 层；注意力把 KV 按 2 倍或 1 倍压缩，再由稀疏索引器为每个查询挑出 512 个压缩位置；路由专家 384 个，每个 token 选 6 个，权重是原生 MXFP4。

问题在于容量。原生权重约 474 GiB（专家 268.9 GiB，Engram 189 GiB），而我们的机器只有两张 RTX PRO 6000 Max-Q（每张 95.6 GiB）和 192 GB 内存，合计约 383 GiB。公开的双卡方案 [3] 把专家量化到 3.0 bpw，速度很快（同样两张 RTX PRO 6000，单流解码 66–74 tok/s，4.6 万 token 提示词预填充约 1 万 tok/s），但相对原版的 KL 散度为 0.0545、top-1 一致率 92.9%，而且需要约 250 GiB 空闲内存，本机放不下。我们选择了另一条路：保留原生 MXFP4，把权重按热度分布在显存、内存和 NVMe 三层（方案 B），在 mx-llama.cpp [2] 上实现调度器 Abyss，再逐项找出瓶颈并优化。

本报告记录这一过程。主要结果见表 1（10 月）和表 2（11 月）：

- **解码**：GPU 专家缓存、解码注意力只取 top-k 行、按热度准入的未命中分流，使解码从 14.1 tok/s 提升到只追加的多轮会话里约 30 tok/s（内存 3600 MT/s）；把计算图和索引器腾出的显存、GPU0 的空余显存都给专家缓存后约 38 tok/s。11 月按剖析逐项去掉串行段和零碎开销：CPU 算的专家晚一层加回残差（专家延后），专家统计不再切图，缓存加了准入门，小内核合并，输出层两卡各算一半，内存提到 6000 MT/s，会话里约 52 tok/s（解码统计每 token 约 18.4 ms），生产上按请求约 50 tok/s。
- **预填充**：CED 感知预填充让解码器层只计算后面还能看到的 token，加上 Engram 预取和官方稀疏索引器，4.8 万 token 提示词从 135 s 降到 22.7 s；再加上两卡流水线、去掉几处主机等待，降到约 14 s，9.6 万 token 约 27 s。大批次分流、末段一遍算完、大批次用缓存、上一轮回答不再重算，把多轮会话每轮的预填充从 5.7–7.2 s 降到约 2 s。
- **和上游比**：同一份权重、同样的启动参数下，上游 llama.cpp 的解码慢约 20%，长提示词的预填充慢 5 倍以上；用三个公开智能体 benchmark 的子集测（不是官方结果），同样的工作量我们用时约为上游的一半（§11）。
- **测法**：同一提示词第二次、一字不差的生成会让专家缓存命中偏高，解码速度虚高约 15–20%；nsys 按节点跟踪会把 CUDA graph 的启动放大约 3 倍，让我们高估了张量并行的收益。我们改用只追加的多轮会话、固定种子和整图跟踪重新测量，并据此修正了此前的结论（§3.3、§3.5）；也给出了 llama-bench 和 AIPerf 的结果，并说明它们为什么和我们的测法不同（§3.6）。
- **正确性**：所有改动都有对拍。我们区分了两档数值底线，对稀疏索引器和解码注意力做

了对官方 PyTorch 实现、对 float64 的逐层对拍，并用强制解码逐 token 比较解码路径的 PPL。唯一改变计算的近似是专家延后，PPL 的差别在误差以内（§3.4）。

- **真实负载**：在 Codex 和 Hermes 的实际请求中发现了 prompt 缓存的选择问题、槽位被只共享开头的请求占用、请求文字里的控制 token 字样被当真、上一轮的回答每轮重算，以及三处只在特定提示词长度下出现的崩溃，均已修复。
- **没有采用的**：专家预取、DSpark 投机解码、每 token 少用一个专家、输出层 Q8_0 和第 0–19 层的张量并行都实现或测过，收益抵不过代价（§4.16、§10）。

2 硬件与模型

2.1 硬件

表 3 是机器的配置和实测带宽。两点决定了后面的设计：内存是双通道、每通道两条，AMD 默认只跑 3600 MT/s，纯读只有约 44 GB/s（后来提到 5200 和 6000 MT/s，纯读约 60 GB/s，§7）；两张卡各走 PCIe 5.0 x8，单卡主机到显存约 28.9 GB/s，3600 MT/s 时两卡同时从主机读合计只有 43.4 GB/s，被内存带宽卡住（6000 MT/s 时 54.5 GB/s，卡在 PCIe）。所以无论专家由 CPU 计算还是上传到 GPU，每读一个专家都要占用同一份内存带宽。另有三项系统设置是必需的：BIOS 里的 IOMMU 要关，开着时两卡之间的直接拷贝会静默出错（§7）；PCIe 固定为 Gen5（§6.3）；系统盘关掉 NVMe 的自动省电（APST，§4.11）。

2.2 模型

图 2 是 DeepSeek-V4.1-Flash 的逐层结构和方案 B 在本机的布局，维度取自 GGUF 元数据。与本报告相关的结构特点有四个：

- **编码器-解码器（CED）**。第 0–19 层是因果编码器，第 20 层的压缩器生成所有解码器层共享的全局 KV。解码器层只通过各自 128 个 token 的滑动窗口读自己的隐状态，因此提示词前面的大部分 token 不需要走完解码器。
- **压缩注意力**。第 0–1 层只有滑动窗口；第 2–19 层把每 2 个 token 压成一行，第 20–39 层每个 token 一行。压缩 KV 只在第 2、8、14、20 层写入，其后的层复用。每个查询有 64 个头，键值只有 1 个 512 维的头（K = V）。
- **稀疏索引器**。第 2、8、14、20、24、28、32、36 层用 32 个头、128 维、FP4 的查询和键给压缩位置打分，选出 top-512；第 20 层还按每 8 行一块挑出 2,048 个候选块，供第 24–36 层重选。其余层沿用最近索引源的结果。
- **专家与 Engram**。每层 384 个路由专家选 6 个，外加 1 个共享专家；每个 MXFP4 专家约 18.8 MB，每层 7.2 GB。第 1、14 层按最近 2–4 个 token 的 n-gram 哈希查两张约 3.84 亿行的大表。

表 1 | 10 月的改动与效果。解码在 4.8 万上下文下测（10-28 的几行是 9.6 万上下文、只追加的多轮会话）；预填充为 4.8 万 token 提示词、Engram 页缓存已热（最后一行是多轮会话的每轮预填充）；KLD 是对改动前的 logits，「—」表示没有改变数值。

补丁	改动	效果	对拍
环境变量	预填充的 MXFP4 矩阵乘改用 8 bit 激活（W4A8）	预填充约慢 5%	0.044 → 0.012*
分支自带	GPU1 上的专家缓存（14 GB，约 780 槽位，LRU）	解码 14.1 → 25.2 tok/s	0.0005
0003	CED 感知预填充：解码器层只算可见 token 的 FFN	预填充 135 → 41.8 s	0.00004
0004	Engram 整批预取	冷缓存 91.1 → 54.3 s	—
0005	解码器层的注意力也只算子集	41.8 → 27.9 s	0.00007
0006	补上官方稀疏索引器（top-512）	27.9 → 22.7 s；修正长上下文	见 §5.3
0008	解码注意力只取 top-k 行	解码 27.7 → 30.6 tok/s‡	0.003†
0009	未命中按热度一半上传、一半交给 CPU	30.6 → 36.1 tok/s‡	0.0003†
0014	预填充大批次分流	每轮 5.7–7.2 → 4.6–5.6 s	0.0005
0015	提示词末段一遍算完：末段不拆调用，最后一个 ubatch 对齐到满	每轮 4.6–5.6 → 2.9–3.5 s	0.009§
BIOS	内存 3600 → 5200 MT/s	解码约 +6%，每轮 2.7–3.2 s	重跑一致
配置	上下文 12.8 万 → 19.2 万，专家缓存 14 → 12 GB	解码约 −3%，预填充不变	0.000005
模板	对话模板按官方 `encoding.py` 重写，采样改为官方推荐值	思考不再越写越重复	—
0017	槽位被只共享开头的请求占用前先存进内存缓存	下一轮重算 26,438 → 30 token	输出相同
权重	换用 1017 版（自己转换）	PPL −3.4%，速度不变	0.075¶
0018	请求文字里的控制 token 字样只按普通文字编码（官方 Recipe 的规则）	不再被当成控制 token	逐 token 一致**
0019	全零张量用到时才分配，没有查询行的层不算查询	GPU0 少用 2.6 GB；预填充快 3–5%	输出不变
0020	索引器和预填充注意力按查询分块，专家缓存 12 → 16 GB	解码 31.95 → 35.42 tok/s	0.0079
配置	第 12 层的 gate、up 专家放到 GPU0，专家缓存 16 → 20.5 GB	解码 35.54 → 38.06 tok/s	输出相同
配置	未命中的上传比例 0.5 → 0.3	解码 37.75 → 38.64 tok/s	约 2e-4†
0023	预填充大批次里缓存已有的专家直接在 GPU1 上算	每轮快 0.3–0.4 s	1.7e-4†

* 预填充路径对解码路径的 KLD，不是对改动前。† server 上 temperature 0 生成时的 top-20 KL；0008 经过编码器层，属于较高的一档底线（§3.1）。‡ 同一提示词第二次、一字不差的生成，缓存命中偏高；只追加的多轮会话见 §4.4。§ 和只换 ubatch 大小的差别一样（§5.5）。¶ 衡量的是权重的变化，不是数值误差（§9）。** 和官方编码器逐 token 一致；不含这类字样时和原编码相同（§6.6）。

表 2 | 11 月的改动与效果。解码是 9.6 万上下文、只追加的多轮会话，每行是那项改动自己那次两档对比（同一编译、开关前后），行与行之间的测试条件不完全相同；对拍是 server 上 temperature 0 生成时对改动前的 top-20 KL，0 表示逐位相同。

补丁	改动	效果	对拍
0025	专家延后：CPU 算的专家晚一层加回残差	解码 36.59 → 40.63 tok/s	1.0e-3
0026	专家统计改成每个 ubatch 异步拷回，不再切图	39.11 → 42.17 tok/s	0*
0027	超连接混合系数的 K 切分	41.97 → 43.86 tok/s	3.8e-3
0028	解码时提前预读要取的 Engram 行	44.15 → 44.63 tok/s	0
0029	CPU 采样的 exp 向量化	采样每 token 0.41 → 0.12 ms	选中 token 相同
0030	缓存的准入门，按衰减热度淘汰	45.32 → 46.75 tok/s	6.1e-4
0032	超连接和 KV 拼装的小内核融合	解码统计 19.8 → 19.3 ms	0
0034	输出层两卡各算一半	47.83 → 48.54 tok/s	0
0035	采样快路径：直接在 logits 上抽	采样每 token 0.23 → 0.07 ms	逐 token 相同
BIOS	内存 5200 → 6000 MT/s	48.32 → 49.19 tok/s	—
0036	矩阵乘向量的尾部每块多算几行	50.96 → 51.99 tok/s	0
0038	大 CUDA graph 分两段启动	51.49 → 52.04 tok/s	0
0039	解码时 NVMe 保活	50.71 → 51.19 tok/s†	0
0040	输出层只分两半加载，专家缓存 20.5 → 21.1 GB	51.77 → 52.04 tok/s	5.5e-4
0041、0042	上一轮回答不再重算：检查点窗口为 0，生成时存检查点	之后每轮 3.02 → 2.46 s	约 1e-3
0043	CUDA graph 空闲后不清	每轮开头约省 15–30 ms‡	0
0044	长提示词预填充两卡流水线	9.6 万 token 40.9 → 28.7 s	0
0045	检查点拆分不再多出小调用	16 条消息、9.6 万 token 36.0 → 30.6 s	8.2e-3§
0046	拷主机上的权重前不再让主机等	智能体之后每轮 2.33 → 1.96 s	0

* 对不开统计；对旧的统计方式是 1.9e-3，差在路由是否融合（§4.9）。† 每档前把模型文件清出页缓存（§3.5）。‡ 估计值。§ 拆法变了，ubatch 的边界跟着变，在经过编码器层那一档底线以内（§5.8）。

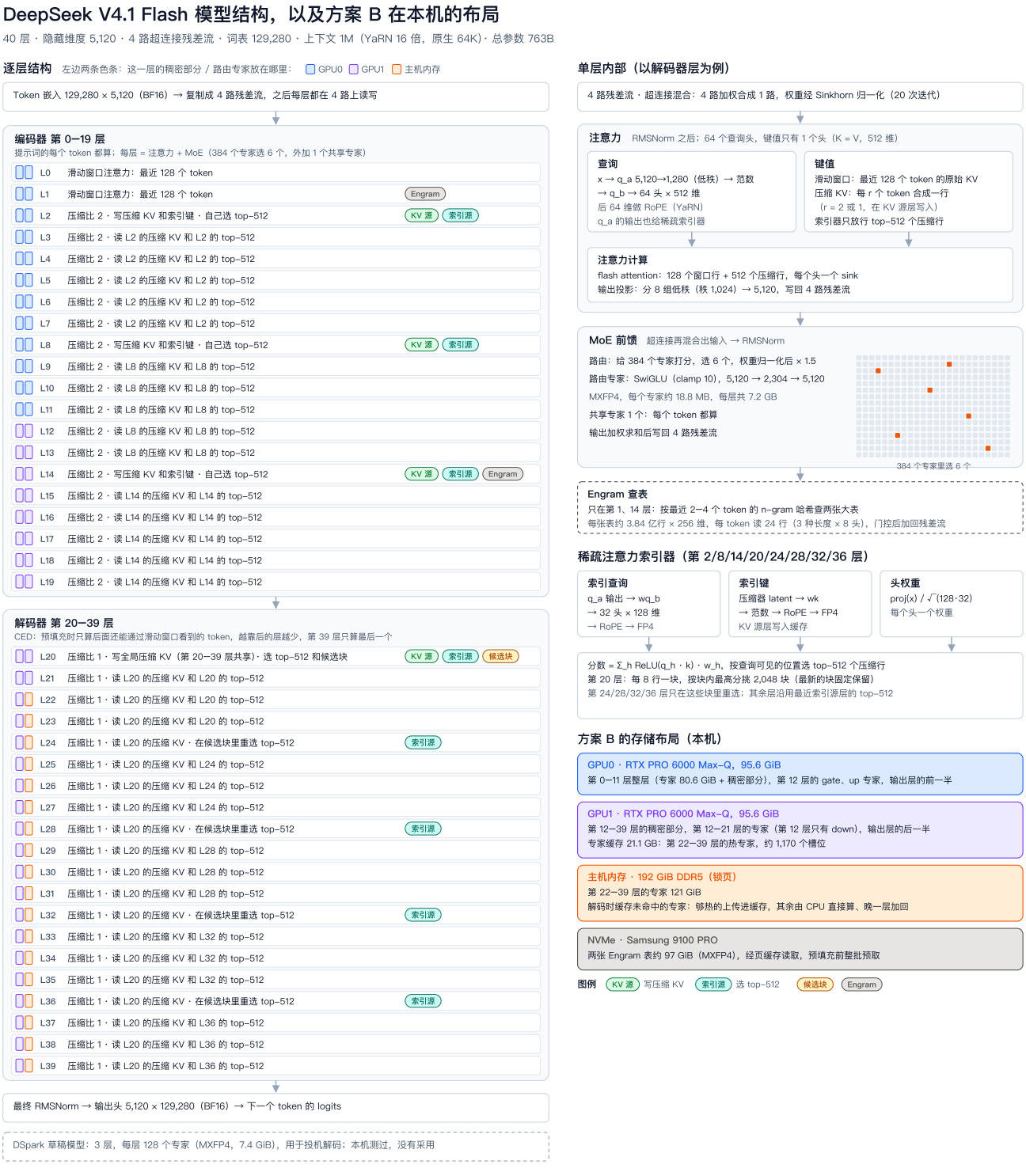


图 2 | DeepSeek-V4.1-Flash 的逐层结构，以及方案 B 在本机的布局。左侧每行一层，色条表示该层稠密部分和路由专家所在的位置；右侧是单层内部、稀疏索引器和存储布局。

表 3 | 硬件与实测带宽（STREAM 与 nvbandwidth；内存在 3600 MT/s 时测于 2026-10-20，5200 和 6000 MT/s 时测于 10-25 和 11-05）。

项目	配置与实测
GPU	2 × RTX PRO 6000 Blackwell Max-Q（300 W），每张 97,887 MiB GDDR7，SM120
互联	PCIe 5.0 x8 / x8，P2P 经 CPU，无 NVLink；卡间单向约 27.7 GB/s
CPU	AMD Ryzen 9 9950X，16 核 32 线程，单 NUMA
内存	4 × 48 GB DDR5（双 rank），双通道 2DPC。纯读、STREAM Copy / Triad：3600 MT/s 时 44、42 / 32 GB/s，5200 MT/s 时 54、53.5 / 39.7，6000 MT/s（现在）时 60、61.1 / 45.2
主机 → 显存	单卡 28.9 GB/s；两卡同时合计 43.4（3600 MT/s）、50.6–53.0（5200）、54.5 GB/s（6000）
主板与系统	ASUS ProArt X870E-CREATOR WIFI（BIOS 1715），Ubuntu 24.04，驱动 580，CUDA 13.1；IOMMU 关，PCIe 固定 Gen5
存储	Samsung 9100 PRO 4TB（PCIe 5.0）、990 PRO 4TB（PCIe 4.0）

2.3 权重格式与量化

本报告的优化都没有对模型再做量化：改动的只是权重放在哪一层，以及哪些 token、哪些行不必计算，每项改动都和改动前对拍（§3.1）。不过我们运行的 GGUF [2] 本身和官方 checkpoint 不完全相同。11-04 我们按转换脚本的规则把 GGUF 的每类张量和官方 checkpoint 逐类核对，抽样反量化后比较（`precision_audit.py`），结果见表 4。

表 4 | GGUF 中各类权重的格式，与官方 checkpoint 对比（11-04 按转换规则逐类核对，抽样反量化后和 checkpoint 的值比较）。

权重	官方	GGUF	大小	精度变化
路由专家	FP4，每 32 个一个缩放	MXFP4	268.9 GiB	无，逐位相同
注意力的 5 个投影、共享专家、索引器的 q_b、Engram 的 wkv	FP8，32 × 32 块一个缩放	Q8_0	6.8 GiB	相对误差 0.55%，0.8% 的非零值变成 0
Engram 表	**FP8，约 189 GiB**	**MXFP4**	**97.3 GiB**	**相对误差 12.0%，9.3% 的非零值变成 0**
嵌入、输出、路由门、压缩器、索引器投影	BF16	BF16	2.6 GiB	无
sink、超连接系数、路由偏置，各种 norm	F32，BF16	F32	0.2 GiB	无

路由专家和原生格式逐位相同。稠密的 FP8 权重转成了 Q8_0，都是 8 bit，只是缩放方式不同，相对误差 0.55%。真正降了精度的是 Engram 表：转换 GGUF 时它从 FP8 压到了 MXFP4，体积减半，也更容易留在页缓存里，相对误差 12.0%，9.3% 的非零值变成了 0。这是 mxxm-t 转换原版时的选择，我们转换 1017 时照着同样的规则做（§9）；NVIDIA 在其 NVFP4 版本中做了同样的转换，测得表的相对 L2 误差也是约 12%，而 GPQA、SciCode 等基准的分

数几乎不变 [7]。本报告的对拍都是同一个 GGUF 改动前后的比较，没有直接测过这一步的影响；要恢复 FP8，得给 fork 加一种「E4M3 加每 32 个一个 E8M0」的类型，表也会变成 189 GiB。

计算时也有几处和官方参考实现不同，多数比官方细。GPU 和 CPU 上的矩阵乘都先把激活按 32 个一组量化成 int8（每组一个 F16 缩放）再乘，即 W4A8；官方用 FP8 激活（每 32 个一个 2 的幂缩放），同为 8 bit。在导出的 600 个路由器输入上，我们的激活量化误差是 0.55%，官方的做法是 2.65%。fork 在 Blackwell 上的预填充默认是 W4A4（激活只有 4 bit），我们改回了 8 bit（§3.1）。残差流用 F32（官方 BF16）。官方把滑窗 KV 和注意力输出舍入到 FP8、压缩 KV 舍入到 FP4，fork 没有这样做，KV 存 F16；稀疏索引器的查询和键用 FP4，这是官方的设计，我们照做。改变计算的近似只有专家延后（§4.8）。

作为对比，方案 A 要把路由专家从约 4.25 bpw 再压到 3.0 bpw，相对原版的 KL 散度为 0.0545 [3]；方案 B 不做这一步，这正是选它的原因。

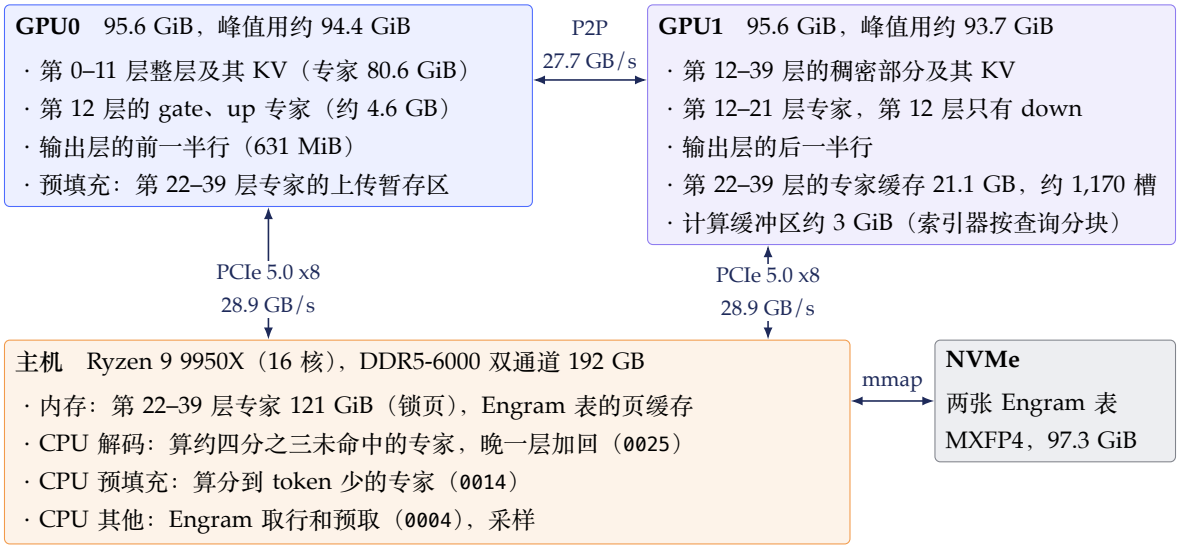
2.4 方案 B 的布局

方案 B 用层切分（`-sm layer -ts 12/29`）：GPU0 放第 0–11 层整层；GPU1 放第 12–39 层的稠密部分和第 12–21 层的专家；第 22–39 层的专家（121 GiB）放在锁页主机内存；Engram 表留在 NVMe，经页缓存读取。编码器的专家全部在显存，所以只要提示词不走解码器层，预填充就基本不碰主机内存。表 5 列出各层的内容。后来加入的 GPU1 专家缓存占 14 GB，大批次分流让 GPU0 的计算缓冲区多占约 1.9 GB；12.8 万上下文时，两张卡分别只剩约 3.6 GB 和 1.2 GB。上下文扩到 19.2 万后，专家缓存降到 12 GB（§8）；10-28 修掉计算图的两处显存浪费、让索引器分块后加到 16 GB，再用 `-ot` 把第 12 层的 gate、up 专家张量（共约 4.6 GB）放到 GPU0 的空余显存，加到 20.5 GB、约 1,140 个槽位（§4.5）；11-07 输出层改成两半分放两张卡之后加到 21.1 GB、约 1,170 个槽位（§4.14）。

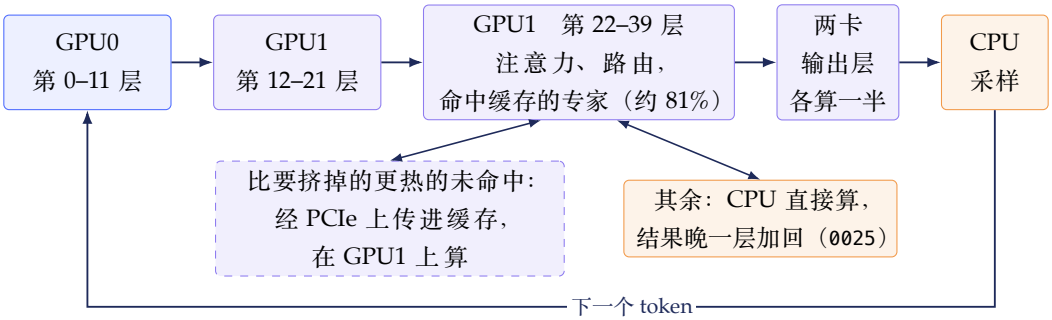
表 5 | 方案 B 的存储布局（现在的配置）。

位置	内容	容量
GPU0	第 0–11 层整层（专家 80.6 GiB + 稠密部分），第 12 层的 gate、up 专家，输出层的前一半行	95.6 GiB
GPU1	第 12–39 层稠密部分，第 12–21 层专家（第 12 层只有 down），输出层的后一半行，专家缓存 21.1 GB	95.6 GiB
主机内存	第 22–39 层专家 121 GiB（CUDA_Host 锁页）	192 GB
NVMe	两张 Engram 表（MXFP4，约 97 GiB）	4 TB

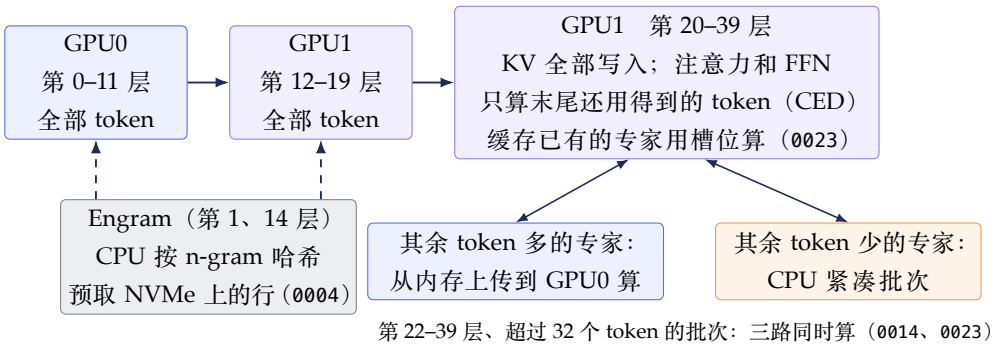
图 3 把放置和运行时的分工画在一起。解码时两张卡串行：GPU0 只算前 12 层；GPU1 算其余 28 层，还要等未命中的专家从内存上传，CPU 同时算大部分未命中的专家，结果晚一层加回残差（§4.8）；输出层两张卡各算一半。预填充时编码器层的全部 token 都在显存里算，解码器层只算提示词末尾还用得到的 token；第 22–39 层的大批次专家沿用 fork 的做法上传到 GPU0 计算，分到 token 少的专家由 CPU 同时算（§5.4），缓存里已有的直接在 GPU1 上算（§5.6）；长提示词的各个 ubatch 在两张卡上流水起来（§5.7）。



(a) 放置与链路（19.2 万上下文、专家缓存 21.1 GB 的配置，§4.14）



(b) 解码一个 token：两张卡串行



(c) 一次预填充调用

图 3 | 方案 B 中 GPU0、GPU1 和主机的分工。蓝色是 GPU0，紫色是 GPU1，橙色是 CPU 和内存，灰色是 NVMe；两卡之间没有 NVLink，P2P 经 CPU 转发。(b) 中两张卡串行，11-08 的生产设置下每 token 约 18.4 ms：第 0–21 层约 6.5 ms，第 22–38 层约 10 ms，第 39 层和收尾约 1.5 ms，token 之间约 0.5 ms（§4.15）。(c) 中编码器层算全部 token，解码器层只算末尾子集（§5.1）；长提示词时 GPU0 算下一个 ubatch 的第 0–11 层，GPU1 同时算这一个的后面各层（§5.7）。

3 评估方法

3.1 数值底线

我们用 llama.cpp 的 KL 散度对拍：先存一份基准 logits，再比较改动后的 logits。早期的编译同一配置重跑完全相同（KLD 为 0）；加了后来的补丁后，重跑之间 KLD 平均约 0、最大约 6e-5，top-1 一致率 100%。重要的发现是数值底线分两档（表 6）：

- 改动只落在解码器层（第 20–39 层）时，KLD 约 0.0005。例如把第 22–39 层的专家从 CPU 改到 GPU 上计算。
- 改动经过编码器层（第 0–19 层）时，0.005–0.01 属正常。例如只把第 5 层的专家改到 CPU 计算就是 0.0064；换内核、关融合、换 ubatch 也在这个量级。

原因在 CED：编码器的输出决定了所有位置共享的全局 KV，那里的舍入差别会扩散到之后所有位置；解码器层的差别只影响当前 token。判断一项改动是否正确，必须先确定它落在哪一档。

同一套方法一开始就查出了一个精度问题：mx-llama.cpp 在 Blackwell 上对 MXFP4 的预填充默认用 W4A4（激活量化到 4 bit），而官方参考实现是 W4A8。预填充路径对解码路径的 KLD 因此高达 0.044、top-1 一致率 91%；改为 8 bit 激活后降到 0.012、95%，与数值底线相当，预填充只慢约 5%。

表 6 | 数值底线与 W4A4 问题（2026-10-21）。

比较	改动范围	KLD
同一配置重跑	—	0（后来最大 6e-5）
第 22–39 层专家从 CPU 改到 GPU	只有解码器层	0.0005
第 5 层专家从 GPU 改到 CPU	经过编码器层	0.0064
预填充 W4A4 对解码路径	全部层	0.044
预填充 W4A8 对解码路径	全部层	0.012

3.2 对官方实现的对拍

KLD 只能说明最终输出变了多少，不能说明是哪一步造成的。对实现复杂的部分，我们写了独立的参照实现（oracle）逐层对拍：用 `llama-abyss-dump` 导出 fork 计算图里指定的张量，再用 PyTorch 按官方 `inference/model.py` 从同样的输入重算。解码步的注意力还用 float64 重算，单独量出内核的舍入误差。具体结果见 §5.3 和 §4.2。

3.3 剖析

性能问题一律先剖析再动手。我们用 nsys 抓取一段预填充或一段解码，按设备统计内核、拷贝和空闲时间，再按 token 平均（`tools/eval/nsys_breakdown.py`、`nsys_decode.py`）。多轮智能体用一个模拟 Codex 的脚本测量：首轮带 4.8 万 token 的文档和两个工具，之后每轮回填约 1,500 token 的工具结果或追问，逐轮记录缓存复用和预填充时间（`agent-turns.sh`）。显

存用 GGML_ALLOC_TRACE=1（0019）记下计算图每次分配和释放的张量，列出计算缓冲区峰值时有哪些张量（alloc-trace.sh、alloc_peak.py）。

剖析 CUDA graph 有一个陷阱。nsys 有两种跟踪方式：按节点跟踪能看到图里的每个内核，但它把图的启动放大约 3 倍（11-06 量到每 token 的 cudaGraphLaunch 合计 2.47 ms，按整张图跟踪只有 0.84 ms）。11-07 以前的解码剖析都按节点跟踪，图启动多的那部分（第 0–21 层，每层都是一两张图）因此偏大，第 0–21 层张量并行的收益就是照它估的，高估了一倍多。现在看图的启动和两卡的空闲用整张图跟踪，看内核才按节点跟踪。主机在哪里等，用调度器和 llama-context 里的 ABYSS_SYNC_TRACE（0044）：它打印每次超过给定毫秒数的主机等待，带源码行、等的是哪张卡和哪个输入，预填充里两处串行的等待都是靠它找到的。

3.4 解码路径的 PPL

llama.cpp 的 perplexity 工具按批处理整段文本，走的是预填充的路径；专家延后这类只在单 token 解码时生效的改动，它测不到。我们给 llama-abyss-dump 加了强制解码（ABYSS_DUMP_FORCE）：提示词之后不采样，而是逐个喂入参考文本的下一个 token，记下模型给它的负对数似然，每个 token 都走单 token 解码的路径。两档在同一段文本上逐 token 配对比较，给出 PPL 的差和它的标准误（defer-ppl.sh、nll_compare.py，约 3,100 个语料和 C++ 代码的 token）。配对比较的误差远小于两次独立测 PPL 的差：专家延后是 +0.06% ± 0.14%，每 token 少用一个专家是 +1.6% ± 0.35%。

3.5 固定种子与冷页缓存

会话测速里 temperature 1.0 的轮次每次生成的内容不同，两档的专家缓存命中率能差 2 个百分点，约合 1% 的解码速度，和很多改动的收益一样大。11-06 起会话测速固定采样的种子（decode_modes.py --seed）：两档的 logits 逐位相同时，整场会话就完全一样，缓存的计数也逐个相同，差别只来自改动本身。另一个干扰是页缓存：同一个会话跑第二档时，Engram 的行已经被第一档读进了内存，读盘的开销偏低，所以每档之前都先把模型文件清出页缓存（DROP_CACHE=1）。

3.6 标准口径

为了和其他系统比较，我们也用 llama-bench 和 AIPerf 测了一遍（std-bench.sh，生产设置、专家缓存 16 GB，表 7）。两者都和本报告的测法不同。llama-bench 的 pp 每个测试只有一次调用，量到的主要是每次调用把第 22–39 层专家从内存读一遍的固定开销（2,048 token 约 3 s），看不出 CED 对长提示词的加速；它用随机 token，tg 的缓存命中 68–73%。AIPerf 单并发、用编程语料、每次输出 1,024 token：首 token 时间就是新提示词的预填充，每 token 时间和只追加的多轮会话相近。AIPerf 生成的提示词开头相同，不关 prompt 缓存时 12–33% 的 token 直接从缓存取，首 token 时间偏低（9.6 万 token 时 24.3 s），所以请求里要加 cache_prompt: false；它的 --cache-bust 只支持 chat 接口。

表 7 | 标准口径的测速（2026-10-28，生产设置、专家缓存 16 GB）。llama-bench 的专家缓存大小靠 `0021` 从环境变量读入；AIPerf 为 0.13 版，单并发。

工具	测试	结果
llama-bench	pp512（深度 0 / 3.2 万 / 9.8 万）	约 250 tok/s
	pp2048（同上）	686 / 678 / 631 tok/s
	tg128（同上）	32.5 / 33.0 / 34.3 tok/s
AIPerf	首 token 时间 p50（输入 1K / 8K / 32K / 96K）	2.70 / 5.53 / 13.25 / 37.1 s
	首 token 时间 p99（同上）	2.79 / 5.76 / 15.4 / 41.8 s
	每 token 时间（输出 1,024 token）	29.4–30.8 ms

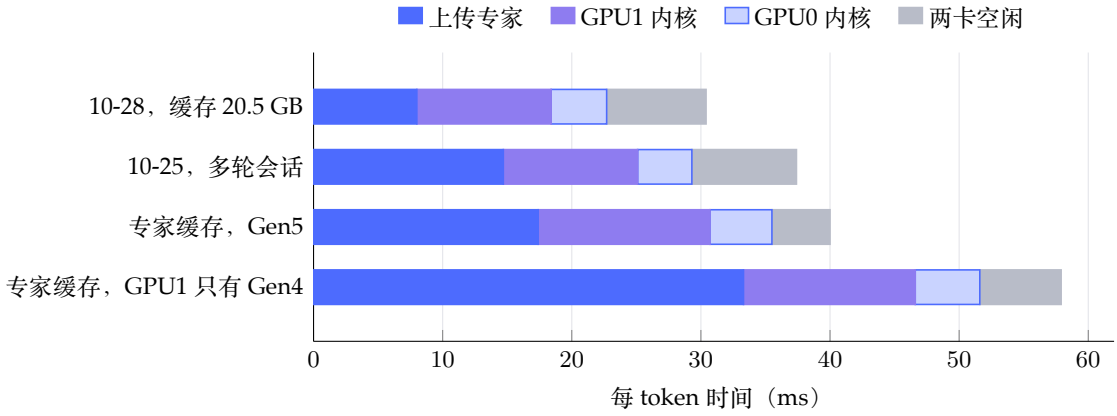


图 4 | 解码的每 token 时间（nsys）。后三行是 4.8 万上下文、内存 3600 MT/s：最下一行是一次开机后 GPU1 的链路只训练到 Gen4，上传慢了一倍（§6.3）；第三行是只开专家缓存时；第二行是全部优化后、只追加的多轮会话中，每 token 37.5 ms（§4.4）。第一行是 10-28 的生产设置（9.6 万上下文、内存 5200 MT/s、专家缓存 20.5 GB），每 token 30.4 ms（§4.6）。

4 解码

方案 B 的原版解码是 16.5 tok/s（空上下文），4 万上下文时约 14.6 tok/s。瓶颈是 CPU：解码器后 18 层每个 token 要读 108 个专家、约 2.0 GB，而内存带宽只有 44 GB/s。

4.1 GPU 专家缓存

mx-llama.cpp 自带一个实验性的专家缓存：在 GPU1 上划出一组槽位（bank），每层一张「专家号 → 槽位号」的映射，命中的专家直接在 GPU 上算，未命中的同步上传并按 LRU 淘汰。我们给它 14 GB（约 780 个槽位，占第 22–39 层专家的 11%）。真实文本的专家局部性很强，LRU 命中率约 75%；4.8 万上下文的解码从 14.1 提升到 25.2 tok/s，短上下文约 29–30 tok/s，KLD 0.0005。

开启缓存后，我们在 PCIe Gen5 下剖析了 4.8 万上下文的解码（图 4）：每 token 40.1 ms，其中 GPU1 上传未命中专家 17.4 ms（约 26.5 个、474 MiB），GPU1 内核 13.3 ms，GPU0 内核 4.8 ms，两卡都空闲 4.5 ms，几部分基本串行。上传是最大的一块。

4.2 解码注意力只取 top-k 行

剖析还显示 GPU1 内核里有约 3 ms 花在 `concat_cont` 上，而且随上下文增长：每层每个 token 都把滑动窗口和整个压缩 KV 拼成一个张量（4.8 万上下文时每次约 40 μs），再由 flash attention 按 top-512 掩码挑行。单个查询时这是浪费的：它只会看到 512 个压缩行。我们改为先用 `get_rows` 取出这 512 行和它们的掩码，再和滑动窗口拼接，补齐到 256 的倍数后走普通 flash attention。

4.8 万上下文的解码从 27.7 提升到 30.6 tok/s，12.8 万上下文从 23.9 提升到 30.9 tok/s（都是同一提示词第二次生成，§4.4），解码速度基本不再随上下文变长而下降。server 上 temperature 0 生成的 top-20 KL 约 0.003，高于解码器层那一档底线。我们用 float64 对拍确认了原因：给定相同输入时，两条路径看到的是同一组 640 行，和 float64 的误差都约 5e-4（F16 舍入）；第 20 层的差别来自上游——第 2–19 层约 5e-4 的舍入差别经 MoE 在分数接近的专家之间选择而被放大，这正是「经过编码器层」那一档的机制。

4.3 按热度准入的未命中分流

原设计是把未命中全部交给 CPU 计算，只按静态热表换入。剖析后发现这行不通：LRU 的高命中率靠的是每个未命中都换入，不换入时命中率会掉到静态热表的水平（每 token 约 47 次未命中），CPU 计算反而比上传还慢；CPU 算完再把同一个专家上传，又要从内存读两遍。

我们改为按热度准入。每层由一个 CPU 算子规划：命中的照常读缓存；未命中的按衰减热度（每层每步乘 0.99）挑出一定比例，立即异步上传进 LRU 槽位；其余由 CPU 用主机内存里的原始专家计算，GPU 那边读 bank 末尾的全零槽位，down 之后两边相加。计算图里 CPU 分支先于 GPU 分支展开，规划算子在 CPU split 开头发出上传，CPU 计算其余未命中时 DMA 正在进行，每个专家只从内存读一遍。

表 8 是上传比例的扫描。只上传一半，命中率不降：按热度挑出来的才进缓存，冷门专家不会把缓存冲掉。最终取 0.5（10-28 专家缓存加到 20.5 GB 后重扫，改为 0.3，§4.5）：4.8 万上下文 36.1 tok/s，12.8 万上下文 36.2 tok/s，KL 约 2e-4，低于解码器层底线。这两个数和表里的速度一样，都是同一提示词第二次生成测出的，偏高；实际会话见下节。

表 8 | 未命中上传比例的扫描（4.8 万上下文，已开启 top-k 收集，复用提示词缓存后生成 256 token，即同一提示词第二次、一字不差的生成）。

上传比例	解码（tok/s）	命中率	对全部上传的 top-20 KL
全部上传（原缓存）	30.6	75.5%	—
1.0（新路径）	30.2	75.5%	0（逐位一致）
0.5	36.1	75.9%	2.7e-4
0.4	35.7	75.1%	2.2e-4
0.3	36.4	74.5%	1.7e-4
0.2	36.0	73.1%	2.2e-4
0（全部交给 CPU）	16.1	0%	1.8e-4

4.4 实际会话中的解码速度

Codex 的实际会话（上下文约 10 万 token）解码只有 30–31 tok/s，明显低于表 8 的 36。差别在测法：表里的速度来自同一提示词的第二次生成，temperature 0 下它和第一次一字不差，第一次用过的专家正好还在 LRU 缓存里，命中率偏高；同一脚本里第一次生成只有 32–35 tok/s。Codex 自带的测速在短上下文下得到 47–49 tok/s，也是同一段生成重复跑出来的。

我们改用只追加的会话测量（`decode_modes.sh`）：首轮放文档和工具定义，之后每轮追加上一轮的回答和一个新问题，生成 256 个 token，轮换贪心或默认采样、流式或不流式。9.6 万上下文 27–31 tok/s，4.8 万上下文 27–29 tok/s，四种方式没有差别，和 Codex 的实际会话一致。所以解码既不受流式和采样影响，上下文从 4.8 万到 9.6 万也不变慢。

表 9 | 只追加的多轮会话中每 token 的时间（nsys，内存 3600 MT/s）。

部分	9.6 万上下文	4.8 万上下文
GPU1 上传未命中专家（约 20 个、360 MiB）	14.5 ms	14.7 ms
GPU1 内核	10.5 ms	10.4 ms
GPU0 内核	4.2 ms	4.2 ms
两卡都空闲	7.9 ms	8.1 ms
合计（nsys 下）	37.1 ms	37.5 ms

表 9 是剖析结果。和上下文长度有关的内核（注意力、索引器打分、top-k、取行和拼接）在 9.6 万上下文时合计只有约 2 ms，top-k 已经是基数选择。一个解码器层的时间线是：GPU1 先算上一层专家的合并、本层注意力和路由（127 个内核，约 0.47 ms）；路由结果和本层输入传回主机后，上传 1 个未命中的专家（3 × 6.3 MB，约 0.70 ms），同时 CPU 算另一个（约 0.79 ms）；之后 CPU 结果传回、启动下一段计算图（约 0.09 ms）。上传和 CPU 计算同时从内存读约 37.6 MB，用了约 0.8 ms，已经到内存带宽的上限；而 GPU 上能挪到上传期间去算的只有本层命中的专家，每层不到 0.1 ms。所以解码剩下的时间主要是每层串行的未命中处理，把上传和命中专家的计算并行起来，每 token 最多省 2–3 ms。

内存提到 5200 MT/s 后（§7），Codex 又用每次不同的短提示词测了一次：生成 128 或 256 个 token，9 次的中位数是 40.3 tok/s（36.8–44.8）。同一时段 Codex 自己的会话（上下文约 10.4 万）是 32.4 tok/s，和表 16 的 31–33 一致。短上下文更快，一部分是注意力相关的开销更少（9.6 万上下文时约 2 ms/token），一部分是生成内容不同、专家缓存命中率不同；到 4.8 万以上，解码就基本不再随上下文变慢。

接入 Hermes 后，长会话（6–9 万上下文，代码和工具输出）里有一半请求的解码掉到 14–26 tok/s。为此我们加了解码统计（`0016`）：每 100 个解码 token 在日志里打印一行，含每 token 耗时、专家缓存命中率、每 token 命中、上传和 CPU 计算的专家数，以及磁盘缺页次数。慢的窗口里命中率、上传数和缺页都和平时一样，也排除了 CPU 被别的进程抢、降频、PCIe 链路代数和语法采样；唯一的相关是忙着的 CPU 核更少，即 OpenMP 线程在层与层之间睡下、再被唤醒。设 `GOMP_SPINCOUNT=100000000` 让线程多空转一会儿，平均从 43.8 降到 35–36 ms/token，超过 50 ms 的窗口从 29% 降到 7–19%。剩下的慢窗口在 10-27 关机、GPU1 出过一次开机故障（§6.3）并重启之后没有再出现：之后 547 个窗口的中位数是 31.1 ms，最

慢的 10% 为 32.8 ms，没有超过 50 ms 的。它们和 GPU1 当时的状态有没有关系，我们没有直接证据。

4.5 腾出显存给专家缓存

解码的速度几乎只取决于专家缓存的命中率。按几次测量拟合，每 token 约 15.9 ms 是固定的，每个未命中的专家再加约 0.45 ms（每 token 在第 22–39 层共挑 108 次）；按这个关系推算，不开缓存约 15.5 tok/s，和最早的实测相符。19.2 万上下文时缓存只有 12 GB（§8），10-28 我们从三处找回显存（表 10）。

计算图的两处浪费（`0019`）。我们给 ggml 的分配器加了 `GGML_ALLOC_TRACE`，记下每次分配计算图时每个张量的分配和释放。`0014` 每层的全零张量是计算图的叶子，图开始前就全部分配（GPU0 上 18 × 240 MiB），改为用到时才生成后，GPU0 加载后从 94,279 降到 91,694 MiB。CED 下没有查询行的层照样算查询，结果没人读，却一直占着显存（GPU1 上 17 × 256 MiB）；去掉后预填充还快了 3–5%（9.6 万 token 从 42.6 降到 41.2 s），输出不变。

索引器按查询分块（`0020`）。GPU1 计算缓冲区的大头是预填充时稀疏索引器的打分表和注意力掩码，大小正比于「ubatch × 上下文」。照 DeepInfer 的做法让打分和注意力每次只算一段查询后，256 个查询一段时计算缓冲区从 6.9 GB 降到 3.2 GB，大段预填充慢 4–8%，对不分块 KLD 0.0079，在编码器那一档底线以内；解码的计算图和不分块完全相同。第一版分块反而更占显存：每层每段各建一个掩码视图，调度器给每个视图都往 GPU 拷一份，同一次切分的输入又在切分开始时全部分配，改为每个计算图共用一组视图才降下来。分块大小扫过 256、512、1024：GPU1 峰值 95,880 / 95,906 / 96,684 MiB，9.6 万 token 提示词 44.0 / 42.8 / 42.0 s；1024 的余量只剩约 1.2 GB，生产用 512。

GPU0 的空余显存。缓存只能放在用它的层所在的卡上：第 22–39 层都在 GPU1，GPU0 上空着的约 6 GB 用不上。我们用 `-ot` 把第 12 层的 gate、up 专家张量（各约 2.3 GB）放到 GPU0，GPU1 腾出的空间加给缓存。第 12 层因此每次多跨卡两次，预填充慢 1–2%；只挪 down 一个张量、缓存 18 GB 时，解码只快 2.9%。

表 10 | 逐步加大专家缓存（2026-10-28，9.6 万上下文、只追加的多轮会话，每档 12 轮）。每一行是同一次测试里改动前后的对比，所以相邻两行的数接不上。

改动	槽位	解码（tok/s）	命中率
索引器分块，缓存 12 → 16 GB	670 → 890	31.95 → 35.42	68.4% → 74.7%
第 12 层 gate、up 放到 GPU0，16 → 20.5 GB	890 → 1,140	35.54 → 38.06	74.0% → 79.0%
上传比例 0.5 → 0.3	1,140	37.75 → 38.64	77.3% → 77.2%

两步加大缓存后，命中率从 68% 升到 79%，解码快了约 19%，和拟合的预测（20.5 GB 时 38.3 tok/s）一致。显存几乎用满：GPU0 峰值 96,386 MiB，GPU1 约 95,900 MiB，余量约 1.5 和 2.0 GiB。9.6 万 token 提示词的输出在三种缓存配置下逐 token 相同；19.3 万 token 时 top-20 KL 约 4e-4，因为缓存大小不同，交给 CPU 算的专家也不同。

缓存变大后，我们又扫了一遍未命中的上传比例（`admit-sweep.sh`）：0.1 / 0.2 / 0.3 / 0.5 时解码 36.44 / 37.83 / 38.64 / 37.75 tok/s，命中率 72.8 / 75.5 / 77.2 / 77.3%。比例太低时缓

存补不上新用到的专家；0.3 的命中率已和 0.5 一样，每 token 却少传约 5 个专家。5200 MT/s 下那次扫描（§7）也是 0.3 略好，生产改为 0.3。上线后（10-28 晚，Hermes 和 Codex 的 53 个请求，上下文 4.6 万到 14.9 万），解码统计的 660 个窗口平均每 token 27.2 ms（36.8 tok/s），最慢的 10% 为 29 ms，没有超过 50 ms 的，命中率约 79%，和测速一致。

4.6 10-28 的剖析

缓存加到 20.5 GB 后（上传比例还是 0.5），我们用生产设置重新剖析了 9.6 万上下文的解码（`profile-prod.sh`，图 4 第一行）。nsys 下每 token 30.4 ms：GPU0 计算 4.3 ms，GPU1 上传未命中的专家 8.0 ms（约 11.5 个、207 MiB，28.6 GB/s），GPU1 计算 10.4 ms，两卡同时空闲 7.7 ms；四段完全串行，GPU1 的上传和计算没有重叠。每 token 有 229 次 `cudaStreamSynchronize`。`nsys_layer.py` 按前后事件给两卡同时空闲分类：约 2.2 ms 是 CUDA graph 的启动（每 token 约 25 次、每次约 90 μs），约 3.8 ms 是没有上传的层里 GPU 干等 CPU 算专家。一层的时间线里，上传挡在这一层的专家计算之前，下一层又要等这一层的结果；能和上传同时做的只有本层命中的专家，边传边算估计每 token 只能省 1–2 ms。

几个便宜的开关也试过（`quick-knobs.sh`、`threads-sweep.sh`，9.6 万上下文、每档 12 轮）。关掉 CUDA graph，解码从 38.99 降到 36.77 tok/s（−5.7%）；开 ggml 的图优化是 39.20 tok/s，在误差以内；都不用。CPU 线程 12 / 16 / 24 / 32 是 38.54 / 37.96 / 37.87 / 36.89 tok/s，差别在噪声以内（命中率本身有约 2 个百分点的起伏），保持 16：没有上传的层里 GPU 等的是 CPU 读专家，每个约 18 MB，16 个线程已基本占满内存带宽。每个未命中的专家从上传改给 CPU 算只省约 0.12 ms（上传约 0.66 ms，CPU 算也在关键路径上），所以把上传全挪到后台，估计也只能再快 5–10%。

4.7 专家的预测与预取

缓存的命中率还有多少余地、未命中能不能提前上传，都要先看真实解码时每层选了哪些专家。11-02 我们给 `llama-abyss-dump` 加了按温度采样，按生产设置在思考模式下解码一个写代码的任务，temperature 1.0、4,000 个 token，每步导出第 18–39 层的路由器输入和选中的专家（`expert-trace.sh`，生产只停了不到 4 分钟）。离线按 sqrt-softplus 加偏置重算路由，和导出的 top-6 一致 98–100%，说明这份记录可以拿来离线模拟（`expert_predict.py`）。

跨层预测。用第 T 层的路由门作用在第 $T-k$ 层的路由器输入上，就能预测第 T 层的专家：$k=1$ 时预测 6 / 8 / 12 / 16 个，覆盖实际专家的 75% / 83% / 90% / 93%；$k=2$ 降到 66 / 74 / 82 / 86%，$k=3$ 是 60 / 67 / 76 / 80%。同一层上一个 token 的 6 个专家只覆盖 33%，所以 token 之间的局部性远不如层与层之间。

缓存策略。1,140 个槽位时，生产的策略（按热度只放三成未命中、LRU 淘汰）命中 80.8%，和导出时的实际值（78–81%）相符；允许不把未命中放进缓存的理论最优（Belady）是 91.2%，余地不小。策略的改进见 §4.12。

预取。在生产策略上按第 $T-1$ 层的预测提前上传缓存里没有的专家，模拟下预测 3 个时每 token 的未命中从 20.9 降到约 15。实现（`0024`，`ABYSS_MOE_PREFETCH`）在第 22–38 层用下一层的路由门作用在本层的路由器输入上选 top K（和路由同样的建图，CUDA 照样融合成一个内核），缓存里没有的挤掉最久没用的槽位，用 GPU1 上第二条 CUDA 流提前上传，下

一层的规划让主流等它的事件。预测本身是准的：传上去的 88–94% 被读到，命中率从 78% 升到 82–85%，每 token 未命中少 4–8 个，9.6 万 token 提示词之后的 32 个 token 和不开时相同。但三种发法都没有让解码变快（表 11）。

表 11 | 专家预取的三种发法（`0024`，9.6 万上下文、只追加的会话，每档 12 轮）。每一行是同一次测试里的四档，第一档不开预取；各行的对照不同。

发预取的时机	每层预测几个（最多传几个）	解码（tok/s）
在规划里发，和 CPU 算专家同时	0 / 2 / 3 / 4	36.27 / 35.74 / 34.25 / 31.88
CPU 算完本层专家后发	0 / 2 / 3 / 3（2）	37.71 / 34.44 / 33.46 / 33.83
GPU1 这一层的计算提交后发	0 / 2（1） / 3（1） / 3（2）	38.59 / 38.47 / 38.57 / 38.45

nsys 下看到了原因（`prefetch-profile.sh`、`nsys_streams.py`）。主机到显卡的拷贝不分流、按提交顺序排队：第二种发法里 36 MiB 的预取挡在下一段计算要的 0.1 MiB 小拷贝（槽位号和 CPU 的结果）前面，GPU1 干等 1.28 ms，预取和计算的重叠是 0%。第三种发法给调度器加了启动回调，等 GPU1 这一层的计算提交后再发，重叠到了 43%；但第 22–39 层每层 GPU1 只算约 0.32 ms，传一个专家要 0.67 ms，其余部分落进了 CPU 算专家的时间：两卡同时空闲从 10.9 降到 8.1 ms，GPU1 忙的时间却从 15.2 涨到 17.8 ms，两相抵消。

三种发法都符合同一个关系：每 token 约等于 GPU 计算的约 16 ms，加从内存读出的专家字节除以约 54 GB/s。CPU 算专家和往显卡传，读的是同一份内存，预取只是换了一条路，总字节没少，只有落在 GPU 计算时间里的那部分才省得下来。往前看一层最多藏住一半左右，再往前看准确率又降一截。按这个关系，与其提前上传，不如让 CPU 算的专家和 GPU 的计算同时进行，这就是下一节的专家延后。预取没有上线。

4.8 专家延后

KTransformers [10] 让交给 CPU 的专家晚一层再合进残差，GPU 不用等它。DeepSeek V4.1 的残差是 4 份超连接的副本：每个子层先把它们混成一份输入（hc_pre），算完再按系数写回（hc_post）。我们照这个思路做了 `0025`（`ABYSS_MOE_DEFER=1`）：单 token 解码时，第 22–38 层交给 CPU 的专家不并进本层的 FFN 输出，它们的加权和 D_l 在下一层注意力的 hc_post 之前，按本层 FFN 的 post 系数加进残差（残差 $+\,\text{post}_l \otimes D_l$）。残差本身是精确的，近似只在于下一层的注意力（包括它的超连接系数）没看到这部分。第 39 层后面没有层可以延后，它的 CPU 专家照常同步算。

计算图的顺序随之改成：GPU1 先算本层命中缓存的专家和下一层的注意力，CPU 同时算未命中；规划算子顺带把 FFN 的输入拷到主机，CPU 不用再等 GPU。调度器也加了一个开关：GPU 段从 CPU 段来的小输入（不超过 1 MiB）走锁页暂存区、在 GPU 的流上异步拷，紧跟 GPU 段、没有输入的 CPU 段也不再同步，主机可以一边算专家一边往 GPU 提交后面的计算。

同一编译开关两档（`defer.sh`）：9.6 万上下文只追加的会话从 36.59 提到 40.63 tok/s（每 token 25.7 降到 22.5 ms，+11%），短上下文（约 2K token）的语料片段从 37.4 提到 45.5 tok/s（+21%）。数值上，9.6 万 token 提示词之后的 32 个 token 全同，top-20 KL 平均 1.0e-3；8 段

语料各 128 个 token 的相同前缀 16–128 个，KL 平均 1.45e-3、中位数 2.2e-4，分岔处是意思等价的措辞。KL 只说明分布变了多少，为了看近似是否损害质量，我们又测了解码路径的 PPL（§3.4）：3,179 个语料和 C++ 代码的 token 上从 3.1934 变为 3.1955，+0.06% ± 0.14%，在误差以内；作为对照，每 token 少用一个专家是 +1.6% ± 0.35%（§4.16）。预填充不受影响。

上线后生产却没有变快（27–33 ms/token）。原因是生产开着专家热度统计：它经调度器的逐节点回调读路由结果，那条路径每段算完都无条件同步 GPU，GPU 先跑的那段也被等完，延后就没了重叠；测试脚本都不开统计，所以没测出来。改成回调什么都不读的段不再同步后，开着统计重测是 35.38 提到 39.31 tok/s（+11%）。生产上 12 个请求（7.7–9.7 万上下文）的 109 个解码窗口平均 24.3 ms/token（41.1 tok/s），上线前是 27.2 ms。

4.9 专家统计不再切图

延后之后我们按内核逐个看了剖析（`kernel_bw.py`、`nsys_kernels.py`）。量化矩阵乘 `mul_mat_vec_q` 在两张卡上达到峰值带宽（1,792 GB/s）的 71% 和 73%，BF16 的输出层一次 811 μs、91%，余地不大。看内核时间要注意一点：开着 PDL（programmatic dependent launch）时，内核会提前启动、在里面等前一个内核，nsys 里单个内核的时长偏大，要按「本内核结束减去前一个内核结束」算它自己的时间。

最大的一项不在内核里。专家热度统计让路由的 top-k 融合内核（3 μs）退回 argsort（14.3 μs，没有 PDL）加一串没融合的小内核，每 token GPU 多 0.76 ms；统计的逐节点回调还在每层的选择结果处切开计算图，每层多一次同步、一次 CUDA graph 启动（每 token 从 25.7 次涨到 82.3 次，每次约 42 μs 的主机时间）。`0026` 给 `llama_context_params` 加了一个回调参数：设了它，选择专家之后给每层的结果加一个输出，每个 ubatch 算完和 logits 一样异步拷进锁页内存，读 logits 时交给回调，不再装逐节点回调。生产设置、开着延后，9.6 万会话从 39.11 提到 42.17 tok/s（解码统计 23.6 降到 21.7 ms），不开统计是 42.37；短提示词的贪心生成快 13%。新统计和不开统计时输出逐 token 相同。

顺带查出旧统计的第 22–39 层是错的：回调按名字前缀匹配，缓存规划和大批次分流对选择结果做的 reshape 和 cont 也被计数，预填充算了两遍、解码每步多算一遍，大批次的一维视图还被当成解码，挑选数是整批的 6 倍；第 0–21 层两边逐层一致。所以开专家缓存以来统计文件的第 22–39 层都不可信，上线时换了新文件。

4.10 GPU 上的零碎开销

统计修好以后，矩阵乘之外的零碎内核和 CUDA graph 的启动在 GPU 时间里变得显眼。我们依次做了四项，都只改单 token 解码；除了第一项经过编码器层、KL 在那一档的底线以内，其余三项的结果和改动前逐位相同。

超连接混合系数的 K 切分（`0027`，`ABYSS_HC_KSPLIT=1`）。每个子层先对展平的残差（4 × 5,120）做不带权重的 rms_norm，再乘 24 行的混合系数矩阵，两步约 12.3 μs、每 token 80 次。它等于 $\mathrm{rsqrt}(\overline{x^2}+\epsilon)\,(Wx)$，CUDA 后端认出这一对就换成两个内核：第一个每块算一段列的点积和这段的平方和，第二个按固定顺序加起来再乘缩放，结果和调度无关。两步合计 3.3 μs；连同统计的修复，GPU0 内核从 4.48 降到 4.00 ms/token、GPU1 从 11.02 降到 9.96

ms，会话从 41.97 提到 43.86 tok/s（+4.5%）。8 段语料的 KL 平均 3.8e-3，和 `0008` 同属经过编码器层的一档。

小内核融合（`0032`）。单 token 解码每 token 约 2,900 个内核，矩阵乘以外的约 3.9 ms。超连接这条链（混合系数之后的 post gate、Sinkhorn、下一子层的 pre gate、合并各份和 rms_norm）并进 K 切分的归约内核和一个合并内核；注意力的 KV 拼装（top-k 取行、转 F16、拼在窗口后面，掩码取行、复制、转换、拼接，7 个内核约 0.53 ms）合成一个内核。每 token 少约 290 个内核，GPU 内核时间少 0.57 ms，解码统计从 19.8 降到 19.3 ms。第一版的 KV 拼装融合结果是错的：原图里取完掩码的列之后，掩码输入就没人用了，分配器把它的内存给了拼接后的掩码，分开跑没事，融合内核边读边写就错；单元测试每个张量单独分配，测不出来。改成逐一核对内核读的张量和两个输出的地址范围，有重叠就不融合。

矩阵乘的尾部（`0036`，`ABYSS_MMVQ_TAIL=1`）。我们给单元测试加了本模型解码形状的性能用例（32 个专家全选，读的数据超过 L2），MXFP4 专家的 down（5,120 × 2,304）只到峰值的 62%：128 线程一块、按块切 K，每行 72 块，第一轮 64 块，第二轮只剩 8 块。让 K 循环最后一轮空闲多的矩阵乘一块同时算几行（每行的求和顺序不变），微基准里 down 从 179.5 降到 143.9 μs；只算 6 个专家、gate 和 up 融合时网格太小，一块多行反而慢，就排除在外。生产剖析里矩阵乘向量每 token 少约 0.24 ms。

CUDA graph 分两段启动（`0038`，`ABYSS_GRAPH_CHUNK=128`）。之前的剖析都按节点跟踪，这种跟踪把图的启动放大约 3 倍；按整张图跟踪，每 token 46 次启动、主机 0.84 ms，最大的两张图（GPU0 的第 0–11 层、GPU1 的第 13–21 层，各 400 多个内核）每次启动 150–170 μs，全在关键路径上。超过 512 个节点的图在第 128 个节点之后切成两张，第一段跑的时候发第二段，两张大图开头的空等从 280、295 μs 降到 137、180 μs，会话从 51.49 提到 52.04 tok/s（约 +1%）。一路按 128、256、512 个节点切的第一版更慢：每个切口 GPU 空 5–12 μs，每层的小图也被切开，把收益抵掉了。

4.11 解码时的 Engram 读盘

Engram 在第 1、14 层，单 token 解码时每层从 mmap 的表里取 24 行，图里的 CPU 取行单独成一段，行不在页缓存里就要等 NVMe。剖析里 GPU0 在第 1 层的 Engram 投影之前平均空等 962 μs（中位数 640），GPU1 在第 14 层前 278 μs，每 token 合计约 1.2 ms，都在关键路径上；`0004` 的预取只管 64 个 token 以上的批次。我们在两张表上实测了几种读法（`engram_reads.c`，每次约 34 行不在页缓存）：逐行去碰 2,896 μs，即每个冷行约 86 μs，这就是原来的取行；先对这 48 行发 `MADV_WILLNEED` 再碰是 184 μs；16 个线程并行碰是 387 μs。`0028` 在算出行号后马上发 `MADV_WILLNEED`，GPU 的空等降到合计 0.49 ms/token，输出逐 token 相同。

剩下的 0.49 ms 里 84% 来自少数 token 各停约 9.2 ms，那是 NVMe 的自动省电（APST）：9100 PRO 空闲 100 ms 就进入一个退出要约 9.5 ms 的状态，页缓存热时连着几个 token 不读盘，下一次冷读就撞上它。关掉这块盘的 APST（运行时写 `pm_qos_latency_tolerance_us`，开机靠 udev 规则）后，空闲多久首读都是约 600 μs。再往细测：空闲 5 ms 以内首读 73–97 μs，10 ms 约 280 μs，15 ms 以上 560–650 μs。一个 token 约 19 ms，冷行每 token 只在开头读一次，次次撞上最慢那档。`0039` 在单 token 解码期间让一个后台线程每 4 ms 用直接 I/O 读一次表文件里随机的 4 KiB，盘就一直不睡（`ABYSS_ENGRAM_KEEPALIVE_MS=4`）。交替四档、各 19

轮，会话从 50.71 提到 51.19 tok/s（+1%），offcputime 里主线程的读盘等待从每秒 14.1 ms 降到 0.3 ms，代价是解码时每秒约 1 MB 的读。生产上冷行很多：两张表 104 GB，页缓存只装得下约 52 GB，bpftrace 统计每 token 平均从盘上读 24.5 页，67% 的 token 至少读一页。

4.12 缓存的准入门与淘汰

在 §4.7 的路由记录上，我们把每 token 的上传数也算进来，模拟了几种准入和淘汰的组合（`cache_policy.py`，表 12）。只把淘汰从 LRU 换成「衰减热度最低」，命中率从 80.5% 升到 81.4%；再加一道准入门「比要挤掉的那个热才放」，最多仍放三成，衰减系数 0.99 时命中率不变、每 token 的上传从 6.3 降到 4.7 个。衰减 0.999 太粘（77.6%），准入门配 LRU 淘汰更差。

表 12 | 在 11-02 的路由记录上模拟的缓存策略（1,140 个槽位，第 22–39 层共用）。

策略	命中率	每 token 未命中	每 token 上传
生产：按热度放三成未命中，LRU 淘汰	80.5%	21.0	6.3
淘汰换成衰减热度最低	81.4%	20.0	6.0
再加准入门，衰减 0.995	80.5%	21.0	2.9
再加准入门，衰减 0.99（上线）	**81.4%**	**20.0**	**4.7**
准入门不限比例，衰减 0.99	82.6%	18.7	7.7

实现（`0030`）：淘汰时扫一遍整组槽位，找衰减热度最低的（空槽位优先，这次规划读到或刚放进的不挤）；准入门不过就停止准入，剩下的交给 CPU。单 token 时 CPU 算的专家，槽位号填 −1，矩阵乘对负的 id 整块写 0，不再去读全零槽位。生产设置四档、各 12 轮会话：原来的生产 45.32 tok/s（解码统计 20.5 ms/token），三项全开、衰减 0.99 是 46.75 tok/s（+3.2%，命中 79.2%，每 token 上传 5.2 个、CPU 算 17.3 个）。衰减 0.995 还快一点（46.95），但被延后的专家更多，热度也看得更久远，生产用 0.99。省下的上传有一部分变成了 CPU 多算的专家，所以比模拟估的约 2 ms 少。对原来的 KL 平均 6e-4，和专家延后同一档：交给 CPU、被延后的专家换了一些。

4.13 采样与 token 之间

按请求算的解码速度总比解码统计慢：统计只量 `llama_decode`，不含它返回之后 GPU 的收尾、采样和 server 自己的处理。`0033` 给 server 的每一步加了计时：server 自己的处理（主循环、队列、组 batch、流式发送）每 token 只有约 0.04 ms，回复再长也不增长；`llama_decode` 之外是等 GPU 收尾 0.90 ms（BF16 的输出层 0.81 ms 加 logits 拷回）和采样（temperature 1.0 时约 0.24 ms）。

采样改了两步。temperature 1.0、top_p 1.0 时，采样链里只有 `dist` 在算：对 129,280 个 logit 逐个 `expf`、用 double 累加，再逐个除。`0029` 改成能自动向量化的多项式 exp、分块求和、先按块找到目标再在块内找，只加载词表的微基准里从 0.411 降到 0.117 ms/token，固定种子 2,000 次抽样选中的 token 全同。`0035` 发现生产的参数下，链里除了 `dist` 全是什么都不做的采样器，原路径却每 token 先建一个 129,280 项的候选数组：链里只剩这类采样器加

dist（或贪心）时，就直接在 logits 上抽，temperature 1.0 每 token 从 0.232 降到 0.069 ms，固定种子的请求逐 token 相同。

4.14 输出层分两卡

等 GPU 收尾的 0.9 ms 里，大头是 BF16 的输出层（129,280 × 5,120，1.32 GB）：它整张在 GPU1 上，这时 GPU0 闲着。`0034`（`ABYSS_HEAD_SPLIT=0.5`）在加载时把前一半行复制到 GPU0，单个输出时两张卡各算一半，在 GPU0 上拼起来再拷回主机；每行还是同一个点积，logits 逐位相同。做的时候踩了三处：后一半用的是建图时的视图，视图还没有缓冲区，调度器认不出它是 GPU1 上的权重，把这一半的矩阵乘也分到了 GPU0，每 token 拷 631 MiB；用了 `-ot` 就不开流水线并行，调度器没有事件，换卡时主机要同步上一张卡，两半成了串行；跨卡拷贝前「等目标卡用完这块内存」也是一次主机同步。调度器为此加了 GPU 接另一张 GPU 时不同步、改在 GPU 上等事件的路径。结果是每 token 等 GPU 从 0.91 降到 0.53 ms，会话从 47.83 提到 48.54 tok/s。代价是 GPU0 多占 631 MiB，加载后只剩约 0.7 GB，桌面不能再用 GPU0。

这之后 GPU1 只算后一半，前一半只在一批有多个输出时才用，所以 `0040` 让输出层不再整张加载，两半各自从文件读到两张卡上（`ABYSS_HEAD_SPLIT_ONLY=1`），每批的输出数限成序列数（server 本来就是 1）。GPU1 少占 630 MiB，给了专家缓存，从 20.5 GB 加到 21.1 GB、约 1,170 个槽位：会话从 51.77 提到 52.04 tok/s，命中率从 80.6% 升到 81.3%。MTP 块和 DSpark 的草稿模型要读整张输出层，不能同开。

4.15 现在的时间分布

11-08 的生产配置下，解码统计约 18.4 ms/token，按请求算约 50 tok/s。每 token 的时间是：第 0–21 层约 6.5 ms，纯 GPU、两张卡轮流算（按 11-09 张量并行原型的实测反推，nsys 按节点跟踪时量到的 8–9 ms 偏大）；第 22–38 层约 10 ms；第 39 层和收尾约 1.5 ms；token 之间约 0.5 ms。

第 22–38 层的时间由主机内存带宽和每层的串行段决定。11-04 的剖析里，这 17 层每层从一次规划到下一次是一个窗口：没有上传的窗口每 token 12 个、每个 0.58 ms，有一次上传的 4.8 个、每个 1.13 ms。上传的 0.66 ms 和 GPU1 的计算排在同一条流上；有上传的窗口里 CPU 和 PCIe 一起读主机内存，合计约 61 GB/s（CPU 约 32、上传约 29），已到内存带宽的上限，CPU 自己的专家本来就和上传一样长。11-07 的剖析里，没有上传的层每层 0.547 ms：GPU1 的内核 0.306 ms，主机自己算（规划加 CPU 的专家）0.334 ms，主机等 GPU 0.133 ms。路由算完约 70 μs 后 GPU1 开始算命中的专家，算完时 CPU 的结果正好到。两卡同时空闲的约 3.8 ms 里，约 1.9 ms 是第 22–38 层每层 CPU 比 GPU1 那段长的约 100–130 μs，约 0.3 ms 是第 39 层不能延后的 CPU 专家，每层规划和启动约 45 μs。

token 之间的约 0.5 ms，在主线程上有 0.41 ms，能分出来的几项是：采样 0.14 ms（测试请求带 `ignore_eos`，走慢路径；生产约 0.07 ms），Engram 预读的 `madvise` 和内核预读约 55 μs，取行反量化约 15 μs，其余输入准备合计约 50 μs，没有哪一项是大头。

所以就算把主机内存的流量全藏进 GPU 计算里，每 token 也还要约 15 ms（约 67 tok/s）；要再快，只能减少 GPU 的计算（第 0–21 层）或者减少从内存读的专家。

4.16 测过但没有采用的

每 token 少用一个专家。 Strata [4] 在四张 R9700 上默认的「Faster」模式每 token 只用 5 个专家。我们用 `--override-kv deepseek41.expert_used_count=int:5` 试了（`experts5.sh`），选中专家的权重照常归一化。4.8 万上下文、只追加的会话里，解码从 32.6 提到 36.8 tok/s（+13%）：丢掉的是排第 6、较冷门的专家，未命中少了 26%，命中率从 65.8% 升到 69.5%；每轮的小预填充也快约 10%。但对 6 个专家的 KLD 是 0.032、top-1 一致率 91.9%，PPL 从 5.863 升到 5.954（+1.6% ± 0.35%）：是数值底线的 3–6 倍，约为 3.0 bpw 量化（KL 0.0545）的六成，抵掉了换 1017 的 PPL 改进的一半。Strata 在 2.25 bpw 上测不出差别，是因为它的量化误差（KL 0.107）盖住了。生产仍用 6 个。

DSpark 投机解码。 fork 的 `draft-dspark` 支持 V4 风格的草稿模型，1017 的侧车（§9）可以直接用（`dspark.sh`）。6.4 万上下文、贪心写代码时，解码从 31.1 提到 36.2 tok/s（+16%），每次验证平均接受 4.3–4.5 个 token，草稿接受率 66–70%。收益的上限很低：一次验证要主模型算 6 个 token，用到的专家更多，未命中成倍增加，约 120 ms，是单 token 的 3.8 倍，5 个草稿全部接受也只快约 1.6 倍。另有三个问题。草稿模型的权重 7.6 GB、计算缓冲区 2.3 GB，专家缓存要降到 6 GB 才放得下，这本身就让 4.8 万上下文的解码从 32.6 降到 28.0 tok/s，抵消了收益。草稿模型要读第 37–39 层的隐状态，预填充时 CED 跳不过解码器层，4.8 万 token 的首轮预填充从 23.4 s 变成 91.7 s。长提示词还让草稿模型的 KV 位置断档（写到 33,112 后，下一批从 48,222 开始），请求以 HTTP 500 失败。贪心输出和不开投机时在第 104–315 个字符处开始分岔，分岔处是近似等价的选择，来自批量验证和单 token 解码的数值差别。暂不采用。11-07 复核时条件更差：之后腾出的显存都给了专家缓存（21.1 GB），开它要把缓存砍到约 10 GB，还要退掉只分两半加载的输出层（§4.14）；这之后的解码优化（专家延后、K 切分、小内核融合、只取 top-k 行）都只对单 token，验证的 6 个 token 用不上；生产 temperature 1.0 的接受率也比这里测的贪心低。外部 4 张卡上的 +59–76% 是专家全放显存时的数，验证几乎不比单 token 贵。

上传的专家也延后。 专家延后之后，有上传的层里本层命中的专家和下一层的注意力要等上传完才能开始（§4.15）。`0031`（`ABYSS_MOE_DEFER_UPLOAD=1`）让每层准入的第一个未命中不在规划里上传：等命中那段计算提交后，用 GPU1 上第二条流上传，再和 CPU 的专家一起在下一层加回残差。上传有 40% 和内核重叠上了，但会话只快约 1%（46.75 提到 47.23 tok/s），解码路径的 PPL +0.12% ± 0.10%。收益少有两个原因：有上传的层里 CPU 和 PCIe 一起读主机内存，卡住的是内存带宽，不是上传排在计算前面；矩阵乘对 −1 的专家照样每行发一个线程块、进去就退出，第一版每 token GPU1 因此多 0.44 ms。没有上线。顺带试过让线程块按 ids 循环、跳过 −1，输出逐位相同，但线程块少了、并行度低，反而慢（48.32 降到 48.12 tok/s）。

q8_1 激活缓存。 分支自带一个缓存，同一个输入只量化一次，可它按先进先出淘汰，和 CUDA 显存池的后进先出释放冲突，会触发断言，生产一直关着。`0036` 让它用自己的 legacy 池，解码的量化调用每 token 从 426 次降到 322 次，但上线一小时后 GPU0 的空闲显存降到约 230 MiB、还在降，就关回去了：legacy 池释放后留着复用，预填充每层的 token 数各不相同，缓存的大小跟着变。`0037` 让专用池只放不超过 128 KiB 的项，显存和关掉时一样，但收益只有约 0.3–0.5%，没有上线。

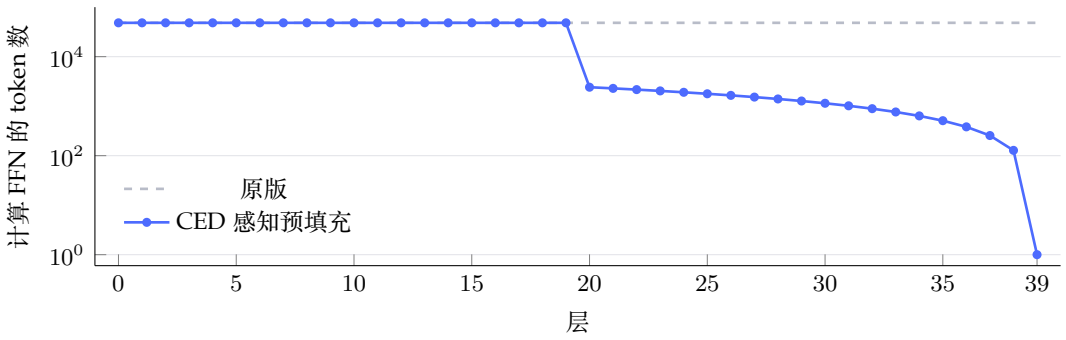


图 5 | 4.8 万 token 提示词每层计算 FFN 的 token 数（按实现中的规划规则计算，没开 DSpark 时不留余量）。编码器层全部计算；解码器层只计算后面还能通过滑动窗口看到的 token。

输出层换成 Q8_0。 BF16 的输出层每 token 读 1.32 GB，换成 Q8_0 是 0.70 GB，权重的相对误差 0.54%（`retype_tensor.py` 只复制含这个张量的分片，在副本里重新量化）。会话快 2.1%（每 token 约少 0.4 ms），top-20 KL 平均 5.5e-4，解码路径的 PPL $-0.01\% \pm 0.02\%$。它不低于 4 bit，但会给原本没有量化的权重引入量化误差，我们没有采用。

三个估过就不做的。把交给 CPU 的未命中临时传到备用槽位、让 CPU 和 PCIe 分担内存带宽：多数层只有 0–1 个 CPU 专家，单个专家走 PCIe（0.66 ms）比 CPU 读（0.38 ms）慢，每 token 只能省 0.23 ms（`cpu_dma_balance.py`）。让缓存偏向不能延后的第 39 层：它的热度乘 4 时第 39 层每 token 交给 CPU 的专家从 1.07 减到 0.54 个，别的层的未命中却多 0.9 个，别的层的 CPU 专家大多也在关键路径上。命中的专家不等主机规划（GPU 上放一张「专家 → 槽位」表，路由一出来就先算命中的）：CPU 那条路才是瓶颈，它要先读回路由、规划完才能算未命中，下一层又要等它的结果，GPU 提前开始只是多等一会儿，估每 token 只省 0.1–0.2 ms，却要几天的工作。

5 预填充

原版对提示词也跑完 40 层：每个 2,048 token 的 ubatch 都要把主机内存里 121 GiB 的解码器专家读一遍。4.8 万 token 的提示词因此要 135 s（约 355 tok/s）。

5.1 CED 感知预填充

官方参考实现对提示词也跑满 40 层，CED 体现在 KV 的共享方式上：解码器层只有滑动窗口用到各层自己的隐状态。因此可以精确地裁剪：第 0–19 层全部 token 都算；第 39 层只算最后一个 token；第 L 层（20–38）只需算最后 $128 + 127 \times (38 - L)$ 个 token——每往前一层，滑动窗口让依赖往前延伸 127 个 token。实现原本在末尾多留 256 个位置，给投机解码的回退用；没开 DSpark 时 DSV4 不能部分回退，后来改为默认不留（§5.5）。这样解码器层合计约 2.4 万个「token × 层」，与提示词长度无关；4.8 万 token 的提示词在原版中是 $20 \times 48{,}240 \approx 96$ 万个（图 5）。

实现分两步。第一步（`0003`）只跳过 FFN，注意力仍对所有 token 计算，KV 写入完全不变；被跳过的 token 之后各层的隐状态不正确，但只会进入没有后续 token 读取的滑动窗口。

每次 `llama_decode` 按自己的末尾规划，所以 server 用 `-b 65536` 让整段提示词进一次调用。4.8 万 token 的预填充从 135 s 降到 41.8 s，8k/16k 上下文对拍 KLD 0.00006 / 0.00004，top-1 一致率 99.8%。

第二步（`0005`）先剖析：此时 41.8 s 里第 20–39 层的注意力占 19.9 s。我们让这些层仍为所有 token 写 KV，但只对 CED 子集取出查询和掩码行计算注意力。预填充降到 27.9 s（1,715 tok/s），KLD 0.00004–0.00007，temperature 0 的输出和第一步一字不差。

5.2 Engram 预取

Engram 表以 mmap 映射且设了随机访问提示，每次缺页都同步读一个 4 KiB 页，2,048 个 token 约需 2.4 s，瓶颈在缺页延迟而不在 SSD 带宽。由于行号只取决于输入 token，我们在 `llama_decode` 拿到整批后立即算出全部行号，由 8 个后台线程按地址排序后用 `MADV_WILLNEED` 预读。页缓存冷时，4.8 万 token 的预填充从 91.1 s 降到 54.3 s，冷读开销从约 39 s 降到约 2 s；热缓存时不变，结果与不预取完全一致。冷读已基本藏在计算后面，io_uring 和按行缓存等后续步骤暂缓。

5.3 补上官方稀疏索引器

fork 原本对 V4.1 的压缩 KV 做稠密注意力。压缩行在 512 以内时与官方的 top-512 选择等价，更长时就不同了：压缩比 1 的层超过约 512 个 token、压缩比 2 的层超过约 1,024 个 token 之后。智能体的上下文几乎都在这个范围里，所以这首先是正确性问题。

我们按官方 `inference/model.py` 补上了完整的索引器：KV 源层写 FP4 索引键（每 32 个一块，缩放 $2^{\lceil \log_2(\mathrm{amax}/6) \rceil}$），索引源层计算 $\sum_h \mathrm{ReLU}(q_h \cdot k)\, w_h$ 并选 top-512，第 20 层挑候选块，其余层沿用最近索引源的结果；flash attention 只看选中的位置。512 token 以内与稠密版 KLD 为 0；8k/16k/32k 上下文时，稀疏版的 PPL 比稠密版低 0.9% / 3.1% / 0.23%（表 13）。由于注意力只看 640 行，4.8 万 token 的预填充也从 27.9 s 降到 22.7 s，12.8 万 token 为 51.8 s（2,464 tok/s）。

表 13 | 稀疏索引器对稠密注意力（perplexity，每档 4 段）。

上下文	PPL（稠密）	PPL（稀疏）	变化	稀疏对稠密 KLD
8k	6.023	5.972	−0.9%	0.057
16k	3.009	2.916	−3.1%	0.094
32k	1.0027	1.0004	−0.23%	0.012

为确认实现与官方一致，我们导出第 2、20、24 层的索引器输入输出，用三种算术重算 top-512 并比较重合率（表 14）：全程 float32；fork 的算术（Q8_0 矩阵乘把激活量化成 int8）；官方的算术（bf16 激活，量化成 e4m3 后做 FP8 GEMM）。fork 与自己的算术重合 99.98% 以上，说明 RoPE/YaRN、FP4、权重缩放、可见性和 top-k 每一步都对；fork 离 float32 的距离（98.7–99.2%）比官方算法自己离 float32 的距离（97.1–98.7%）还近。索引键 99.99% 以上逐位一致，不一致的都是恰好落在 FP4 中点上的单个元素。

表 14 | top-512 集合的平均重合率（4,027 token 提示词）。

层	fork 对 int8 参照	fork 对 float32	官方算术对 float32	fork 对官方算术
2	99.99%	99.16%	98.67%	98.63%
20	99.98%	98.67%	97.14%	97.11%
24	99.98%	98.85%	97.40%	97.36%

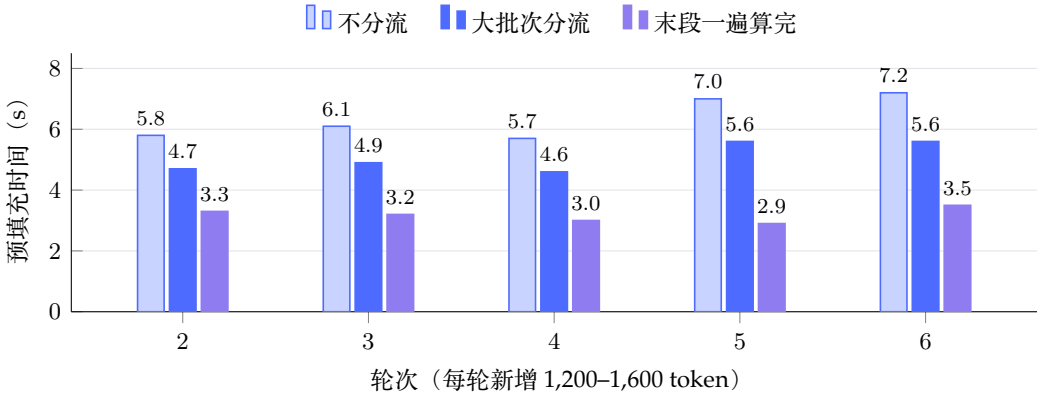


图 6 | 多轮智能体会话中每轮的预填充时间（首轮为 4.8 万 token 的文档，之后每轮回填工具结果或追问；内存 3600 MT/s）。末段一遍算完见 §5.5。

5.4 大批次分流

多轮会话里每轮新增的 token 不多，但只要批次超过 32 个 token，第 22–39 层几乎全部 384 个专家都会被用到，调度器把这些大批次的专家矩阵乘卸载到 GPU0，每层整批上传。一轮 1,610 token 的预填充（7.0 s）中，GPU0 上传 145 GiB 用了 5.4 s，而 CPU、GPU1 的链路和专家缓存都闲着。这部分的下限是把这些专家从内存读一遍，约 3 s。

我们把未命中分流的思路推广到大批次：每层一个 CPU 算子统计每个专家的 token 数，按从少到多把专家交给 CPU，直到 CPU 的预计时间追上其余专家的上传时间；其余照常上传到 GPU0。为此给 CUDA 的 MMQ 补上了 id = −1 的处理（跳过并把输出清零），上传回调直接用规划算子在主机上的结果，不必从 GPU 读回 id。

第一版的 CPU 分支处理整块张量（全部 token × 6 个专家），而 CPU 实际只算约 5% 的行，量化、SwiGLU、清零和 252 MB 输出的回传把收益全吃掉了，比不分流还慢。改为紧凑批次后——只取出交给 CPU 的「token × 专家」（最多 1,024 行），在 GPU1 上取出这些 token，CPU 上按一行一个专家计算，`set_rows` 写回后与 GPU0 的结果相加——2,048 token 一批的预填充从 4.88 s 降到 4.24 s，KLD 0.00046。多轮会话之后每轮的预填充从 5.7–7.2 s 降到 4.6–5.6 s，首轮从 30.6 s 降到 27.9 s（图 6）。

这一步离下限还有约 30 ms/层：GPU0 的上传暂存区在 gate、up、down 三份权重之间复用，每段之间都要同步，CPU 要等 gate 那批传完才能开始（11-09 查明这里的同步是主机在等，见 §5.9）；CPU 计算和 DMA 又共享 44 GB/s 的内存带宽。合并的几步要固定在 GPU0，否则 GPU1 的计算缓冲区会大 3.7 GB，12.8 万上下文时放不下。实现中还发现并修复了 fork 的一处问题（`0013`）：`ggml_backend_sched_reserve_size` 在切分计算图之前重置调度器，清

掉了构建计算图时手动指定的后端，按错误的切分估算计算缓冲区。

5.5 末段一遍算完

CED 让每次 `llama_decode` 调用都按自己的末尾跑解码器层，凡是超过 32 个 token 的调用都要把第 22–39 层几乎全部专家从内存读一遍。server 为了建检查点，会在最后一条用户消息开头、末尾前 4 + n_ubatch 和末尾前 4 处拆调用。生产日志里 Codex 的大多数轮次因此被拆成两次大调用（例如 1,772 + 866 + 4），比同样长度只有一次调用的轮次多花 1.5–2 s；一次调用里末段跨两个 ubatch 时，也要读两遍。我们做了三处改动（`0015`）：

- 提示词最后 2,700 个 token 里不再按用户消息开头和末尾前 4 + n_ubatch 拆调用，只留末尾前 4 个 token 那次，下一轮靠它从助手前缀处接着算。会话的第一个检查点照旧，Codex 的辅助请求靠它复用系统提示。代价是这两类检查点没了：只有改写最后一条用户消息时才用得上，这时 server 退回上一轮的检查点，多算这一轮。
- 一次调用的余数放进第一个 ubatch，最后一个 ubatch 是满的 2,048 个。原来余数在最后，最后一个 ubatch 常常只有几百个 token，第 20–32 层左右都要在前一个 ubatch 里再算一遍；现在前一个 ubatch 只剩第 20–22 层的少量 token。
- 末尾的余量默认等于回退环的深度 `n_rs_seq`，没开 DSpark 时为 0。我们按规划公式逐个位置核对过，余量为 0 时下一个 token 需要的滑动窗口状态仍然完整。

表 15 | 末段一遍算完的效果（`agent-turns.sh`，内存 3600 MT/s）。

配置	首轮 4.8 万 token	之后 5 轮（1,200–1,600 token）
改动前	27.9 s	4.6–5.6 s
末段不拆调用	25.3 s	3.2–3.7 s
+ 最后一个 ubatch 对齐到满	24.4 s	3.2–3.7 s
+ 余量为 0	24.0 s	2.9–3.5 s

新提示词按外部测速的方式（关闭 prompt 缓存，每档 3 次取中位数）测，2,324 token 从 5.57 s 降到 3.69 s，4,277 token 从 8.80 s 降到 4.34 s。对拍方面，最后一个 ubatch 对齐对改动前的 KLD 是 0.0092、top-1 一致率 95.4%，和只把 ubatch 从 2048 改成 1024 的 0.0090、96.0% 一样；余量 0 对余量 256 生成的 256 个 token 全部相同（top-20 KL 4.6e-4）。我们还用 token 数组直接构造提示词，让调用长度除以 2,048 余 0、1、2、7–9、31–33、127–129、1000、2047，各测约 2 千、4 千、2 万 token，42 个请求全部正常。

5.6 大批次用缓存

末段一遍算完之后，每轮的预填充还有一大块固定开销：生产日志里 10-28 下午的 92 次预填充，拟合出每次约 1.84 s，加每个新 token 约 0.98 ms，Hermes 一轮只新增 505 个 token 也要 2.2 s。原以为 CED 让解码器层每轮只算末段约 128 个 token。我们用 `0022` 打印每个大批次用到的专家数和其中缓存已有的（`big-coverage.sh`），实测每轮新增 1,100–2,800 token 时，第

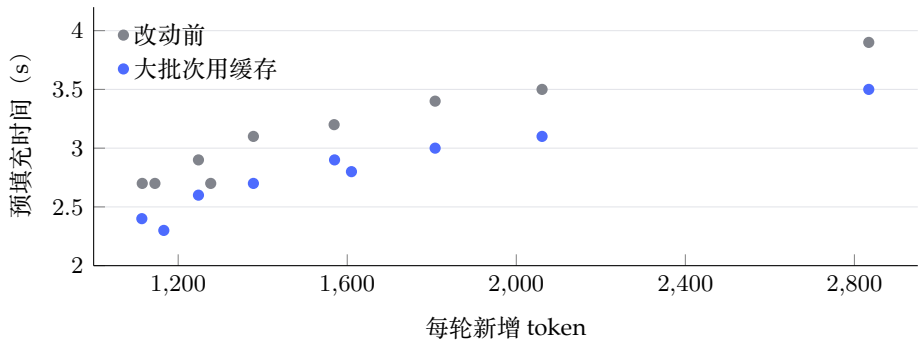


图 7 | 多轮智能体会话中每轮的预填充时间（`big-cache.sh`，首轮 9.6 万 token，之后 9 轮；内存 5200 MT/s、专家缓存 20.5 GB）。两次会话的生成不完全相同，有两轮的新增 token 数不一样。

22–38 层每层每批约 860–1,140 个 token、用到约 240 个专家：CED 下解码器层仍要算大部分新 token。这些专家按个数只有 22% 在 GPU1 的缓存里，按挑选次数却是 55%：缓存里是解码常用的专家，分到的 token 也多。

当时的规划每批把约 126 个专家交给 CPU（其中 13 个在缓存里），116 个从内存上传到 GPU0 算，其中 40 个其实已在 GPU1 的缓存里：缓存只能复制给它所在的卡，大批次却在 GPU0 上算，只好照样从内存上传。剖析一轮（新增 1,609 token，3.2 s）：GPU0 从主机上传专家 37 GiB、用了 1.37 s，GPU0 计算 0.52 s，GPU1 计算 0.61 s（注意力 0.22 s，其中索引器 0.13 s），GPU1 的传输 0.11 s，两卡同时空闲 0.63 s，也基本串行。

`0023` 让规划记下缓存已有专家的槽位，这些专家直接在 GPU1 上用槽位算（gate、up、激活、down，再加权求和），其余的照 `0014` 分给 CPU 和上传到 GPU0。计算图里先发 GPU0 那路，再发 GPU1 这路，最后 CPU 算，三边同时进行，结果相加。同一个会话关、开各跑一遍（`big-cache.sh`，图 7），同样长度的轮次快 0.3–0.4 s（约 11%），整场约 20% 的大批次专家改从缓存算；长的新提示词基本不变，因为缓存是冷的，只命中 16%。GPU1 加载后的显存和峰值都不变，新路径的中间张量和峰值错开了。对拍时先用短提示词贪心解码 512 个 token，把两次的缓存填成一样，再跑 9.6 万 token 的提示词：32 个输出 token 相同，top-20 KL 1.7e-4。

5.7 长提示词的两卡流水线

上游 llama.cpp 有一个还没合并的 PR（#29963），专家在主机内存时也让几张卡流水起来，作者在 3 张 RTX PRO 4000 上测得长提示词快 1.5–2 倍。我们先量了自己的情况（`prefill-pipe.sh`、`nsys_pipe.py`，9.6 万 token 提示词的第二次请求）：40.6 s 里两张卡同时忙只有 0.21 s，GPU0 忙 22.8 s、GPU1 忙 15.0 s。41 个只算编码器的 ubatch 每个 755 ms，其中 GPU0 的第 0–11 层 409 ms，第 12 层在两张卡之间交接 17 ms，GPU1 的其余各层 295 ms，两卡都在等 34 ms（图 8a）。如果 GPU0 算下一个 ubatch 的第 0–11 层时 GPU1 同时算这一个的后面各层，估计约 27 s。

fork 自带流水线（`pipeline_parallel`），但用了 `-ot` 或专家缓存时会关掉，而且它按副本数给每份输入多存一份、整张图期间都占着：按 19.6 万上下文预留时，一份压缩注意力的掩码就是 384–768 MiB，GPU0 要多 1.1 GiB、GPU1 要多 3.3 GiB，放不下。所以我们保留单副本，只去掉串行点。主线程 40.6 s 里有 29.3 s 在 `cudaStreamSynchronize`，用 `ABYSS_SYNC_TRACE` 记

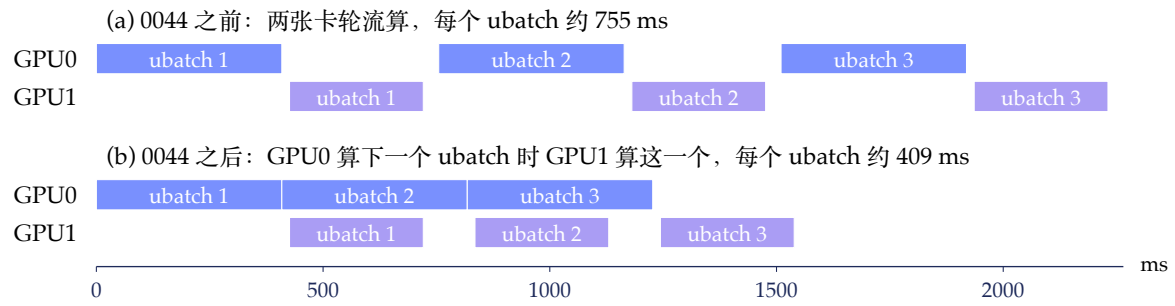


图 8 | 长提示词预填充里只算编码器的 ubatch（2,048 token）在两张卡上的时间线，按 9.6 万 token 提示词剖析里每段的时间画的示意；GPU0 那行是第 0–11 层，GPU1 那行是第 12 层以后的各层。(a) 中 GPU0 每个 ubatch 开头都要等 GPU1 做完上一个；(b) 中两张卡同时忙，每个 ubatch 的时间由 GPU0 的第 0–11 层决定（§5.7）。

下来，主要是三处：往 GPU1 拷请求来的输入（掩码等）之前先等 GPU1 做完（13.8 s）；每个 ubatch 重新分配计算图之前停两张卡（7.9 s：GPU0 上掩码的副本随 KV 增长，超出了上次分配的大小）；从 CPU 往卡上拷之前等目标卡（4.6 s）。`0044`（`ABYSS_OVERLAP=1`）的改法：

- 目标卡还有活排着时，大的输入经锁页暂存区在它的流上按顺序传，不让主机等；卡空闲或输入在 1 MiB 以下照旧直接拷，否则多轮会话每轮要多一次几百 MiB 的内存拷贝。每个输入在每张卡上两个暂存缓冲区轮流用，各有自己的事件；变大的暂存区留到下次同步时才释放，因为释放锁页内存会同步整张卡。
- 新的图布局先只规划一遍，缓冲区不用变大就不停卡：下一个 ubatch 对一张卡显存的写，都排在这张卡已有的活后面。
- 0 字节的输入不拷也不等。输出层的两半（§4.14）之间有一个 0 行的隐状态，它在每个 ubatch 末尾给 GPU0 的流加了一个「等 GPU1 拷完」，而这次拷贝排在 GPU1 这个 ubatch 的全部计算后面，下一个 ubatch 的第 0–11 层又都排在这个等待之后：这是两张卡一直轮流的直接原因。nsys 里内核早 1.5 s 就提交了，GPU0 每个 ubatch 开头仍空等约 280 ms，按提交顺序列出 GPU0 流上的操作才找到它。
- 专家统计交出选择结果原来每个 ubatch 同步一次，改成两个锁页缓冲区轮流交出。

此外要设环境变量 `CUDA_SCALE_LAUNCH_QUEUES=4x`：GPU1 每个 ubatch 有约 1,300 个内核，主机提交时会卡在 CUDA 的启动队列上；只开 `ABYSS_OVERLAP` 是 33.4 s，加上它是 29.0 s。两卡直接访问（`GGML_CUDA_P2P`）对预填充没有作用。

同一编译开关两档、都开着统计：9.6 万 token 提示词从 40.9 降到 28.7 s（−30%），填满 19.3 万从 83.5 降到 58.1 s，智能体首轮（4.8 万 token）从 21.2 降到 15.1 s；之后每轮只有一个 ubatch，流水不起来，2.30 和 2.32 s 一样；会话解码 50.12 和 50.48 tok/s。8 段语料、9.6 万 token 提示词、19.3 万 token 找暗号和 10 轮智能体会话的输出全部逐 token 相同，显存峰值只多 18 MiB。现在卡在 GPU0（图 8b），再快就得挪层，要动 GPU1 的专家缓存。让前面各层两张卡同时算也是一条路，但第 10 节的原型里两张卡一起算前 20 层，这条流水线就用不上了，长提示词反而慢了 8%。

5.8 检查点拆分出的小调用

长提示词按 `--checkpoint-min-step 32768` 在用户消息开头拆调用、建检查点（§6.4）。检查点在组好 batch、调用 `llama_decode` 之前才建，存的是这次调用开头的状态；可组 batch 的循环判断下一刀时，还拿上一个检查点量步长。于是刚过步长、开头要建检查点的这次调用，遇到下一条用户消息又切一刀，成了几百 token、不建检查点的小调用，而在 CED 下每次调用都要把第 22–39 层的专家读一遍。11-08 23:42 重启后的第一个长请求（7.7 万 token）被拆成 33,279、381、32,665、318、10,627 和 4，两次小调用各约 1.7 s；10-20 以来的日志里，32 个请求有 48 次这样的小调用，合计约 109 s。

`0045`（`ABYSS_CKPT_COUNT_START=1`）让循环把这次调用开头的检查点算进去：下一刀量的是离这次开头的步长，§5.5 的末段合并也从它起就生效。同一补丁还让每个请求在日志里多打一行这次提示词拆成了几次调用、各多少 token、在哪里建了检查点；原来默认的日志级别下看不见开头这些小调用。用两段多消息的长对话测（`ckpt_split.py`）：16 条消息、各约 6K 的 9.6 万 token 从头算，从 6 次调用变成 4 次（35,425 + 35,183 + 25,483 + 4），36.0 降到 30.6 s；消息长短不一的 11.2 万 token 智能体会话从 8 次变成 5 次，42.1 降到 36.9 s；改约 60% 处的一条消息后重算，两段各少一次调用，快 2.7 s 和 1.5 s。检查点的位置不变。拆法变了，ubatch 的边界跟着变，前缀相同的 89 个 token 上 top-20 KL 加权平均 8.2e-3，在编码器那一档的底线以内，分岔都在近乎平局的候选上。上游 llama.cpp 的 server 也有同样的判断。

5.9 大批次上传不再让主机等

§5.4 说过，大批次分流离下限还差约 30 ms/层，原以为是 GPU0 的上传和计算没有流水起来。11-09 我们先量（`big-pipe.sh`、`nsys_big.py`，智能体第 6 轮，新增 1,095 token、预填充 2.1 s）：第 22–39 层每层分给 GPU0 的专家分 gate、up、down 三段上传，每段 10–21 ms（29 GB/s），每段之后 GPU0 只算 1–2.5 ms，计算全藏进上传里最多也只省 0.09 s，不值得做。时间其实花在等待上：这 17 层共 1.60 s，其中两张卡同时空着 0.45 s，主机在同步里等了 0.92 s。`ABYSS_SYNC_TRACE` 显示，是调度器拷主机上的权重之前先让主机等 GPU0 做完（每轮 34 次、共 620 ms）：主机卡在里面，CPU 那份专家要等三段都排完才开始，GPU 做完了又等 CPU。

拷贝本来就排在 GPU0 自己的流上、在用到同一块显存的计算之后，按顺序已经安全；读设备数据或改写主机数据的路径（专家缓存查表要读专家 id）本来就自己同步。`0046`（`ABYSS_WEIGHT_NOWAIT=1`）让交给拷贝回调的主机权重不再先等。清了页缓存的两档对比：智能体之后 9 轮的预填充平均从 2.33 降到 1.96 s（−16%，图 9），首轮 4.8 万 token 从 14.7 降到 14.1 s，9.6 万 token 新提示词从 27.6 降到 26.8 s，解码不变（18.5 ms/token）。语料和 9.6 万 token 提示词的输出逐位相同，显存峰值多 16 MiB，那次等待从每轮 36 次、742 ms 变成 0。剩下的主机等待是每层规划之前等 GPU1 算出路由，那是真依赖。

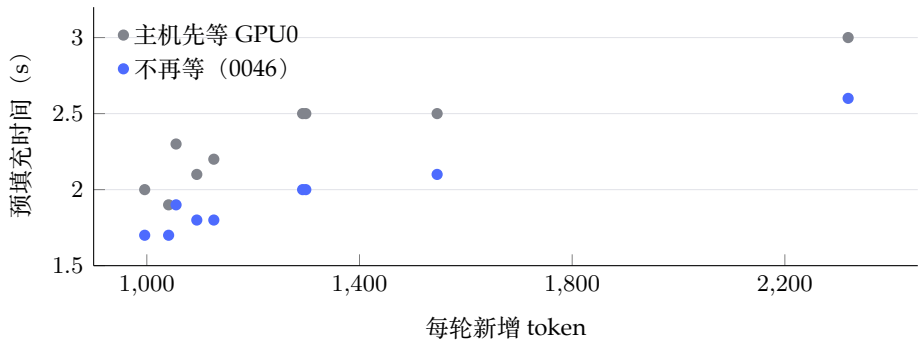


图 9 | 智能体会话之后每轮的预填充时间（`agent_turns.py`，首轮 4.8 万 token，之后 9 轮回填工具结果或追问；内存 6000 MT/s、专家缓存 21.1 GB，两档前都清了页缓存）。两档每轮的输入完全相同。

6 服务与智能体负载

6.1 多轮会话的缓存复用

V4.1 的缓存不能回退到滑动窗口中间，只能退回检查点。server 在每条用户消息的开头、最后一条用户消息开头和提示词末尾附近创建检查点。模拟会话中，从第 3 轮起每轮只重算上一轮生成的回复加 4 个 token，当时以为是回复经模板重新渲染后与生成时的 token 不完全一样，server 退回到「Assistant」前缀处的检查点；11-07 查明主要原因在别处（§6.9）。

6.2 prompt 缓存的选择

在 Codex 的 6 个真实请求中（预填充合计 76 s，生成 129 s），我们发现 server 只有一个槽位，Codex 的辅助请求（与主对话共享约 4,500 token 的系统提示和工具定义，只问一句）会把主对话挤出槽位。server 已把主对话存进内存里的 prompt 缓存，但主对话回来时，槽位里的辅助请求与它仍有 94% 的前缀相同，而 server 只在保留比例低于一半时才去缓存里找，并且按保留比例挑选，长的条目反而落选。

我们改为按「能直接用上的 token 数」比较槽位和缓存条目——对检查点受限的缓存，就是不晚于分叉点的最近检查点的位置——多出 256 个以上就换入；只用到一小部分的条目复制一份，原条目留在缓存里。另一个问题是辅助请求自己也从头算：系统提示末尾有检查点，但 server 要求检查点比分叉点早至少 128 个 token。这是为保存滑动窗口的检查点定的规则；V4.1 的检查点是那个位置的完整状态（`pos_min == pos_max`），在分叉点之前就能接着算。修复后，同一场景里辅助请求从 8.4 s 降到 0.3 s（只重算 13 个 token），主对话的下一轮从内存缓存换回，只重算 216 个 token、2.2 s。

6.3 真实负载暴露的缺陷

- **未分配的输入张量。**CED 让某些层在某个 ubatch 里没有 token，依赖它们的计算被整段丢掉，预先建好的输入张量就不会分配，往里写数据时断言失败。这在 CED 行映射（提示词约 2,500–16,000 token）和候选块输入（约 1.7 万 token 以上、长度除以 2,048 余约 114–622）上各出现一次，都只在特定长度下触发，测速用的 4.8 万和 12.8 万 token 恰好都没碰到，由 Codex 的请求暴露。

- **全零槽位重复。**未命中分流最初只有一个全零槽位。9–32 token 的批次里，同一 token 的多个 CPU 未命中指向同一个槽位，CUDA MMQ 的重排每个专家每个 token 只记一行，漏掉的行号未初始化，量化内核越界。改为每个 token 的第 k 个专家用第 k 个全零槽位。

- **GPU1 开机故障。**四天里 GPU1 在开机时出了四次问题，GPU0 一次也没有：10-24 链路只训练到 Gen4，主机到 GPU1 只有 14.4 GB/s，未命中上传慢了一倍，解码从约 25 掉到 17 tok/s（图 4）；10-25 改 BIOS 后第一次开机只到 Gen3，只有 1.78 GB/s；10-27 关机后再开机，驱动初始化不了 GPU1 的 GSP 固件（`RmInitAdapter failed`、`WPR2 already up`），重启后又只到 Gen4（14.47 GB/s），再重启才正常。测速脚本会先检查链路。10-28 断电时在 BIOS 里把 PCIe 固定为 Gen5（没有重新插卡），之后第一次开机两张卡都是 Gen5 x8；但 10-31 冷开机时 GPU1 又只到 Gen4（14.44 GB/s），热重启一次就恢复，固定 Gen5 没能防住。前后五次都是 GPU1，都在冷开机或改 BIOS 之后，热重启都能恢复；11-02、11-03、11-05 的几次冷开机两张卡都是 Gen5，11-09 前后 GPU1 重插过。所以开机后要先确认 `nvidia-smi` 里有两张卡、链路都到 Gen5。

- **测试 server 退出时卡住。**测试 server 收到退出信号时正好接到外部请求，之后 GPU 和 CPU 都是 0%，只能强制结束。生产重启时也碰到过退出清理卡一分钟以上；重启脚本现在等 60 s 后再发一次信号（槽位已先存好，第二次会立即退出），再等 30 s 仍没停就强制结束，然后等驱动把显存还回来再启动，否则紧接着加载会分配显存失败。

- **Codex 的 MCP 工具被丢弃。**Codex 每个请求带的 9 个 `namespace` 类型工具，server 不支持，直接跳过，模型看不到。这个问题还没有修。

6.4 重启后的长对话首次预填充

server 重启后 prompt 缓存是空的，Codex 的下一个请求要把整段对话从头算。2026-10-25 的日志里，一段约 10.4 万 token 的多轮对话用了 85.2 s（1,219 tok/s），而单条消息的 12.8 万 token 提示词只要 51.8 s（2,464 tok/s，§5.3）。差别来自检查点：server 在对话中间每隔至少 8,192 个 token（`--checkpoint-min-step` 的默认值）、在用户消息开头处拆一次调用并建检查点，这段对话因此被拆成至少 16 次调用。每次调用都按自己的末尾跑解码器层，要把第 22–39 层的专家再读一遍；§5.5 的合并只管最后 2,700 个 token。

我们从两头改。一是重启脚本在停 server 之前把槽位 0 连同检查点存盘（`--slot-save-path`），启动后读回：9.6 万 token 的槽位文件约 0.34 GiB，存 86 ms、读 114 ms，重启后的第一个请求接着算，不用重算整段对话。二是把 `--checkpoint-min-step` 调到 32768，没有存档时（例如崩溃后）拆的次数少很多：9.6 万 token、16 条用户消息的对话首次预填充从 77.3 s 降到 52.9 s（`restart-prefill.sh`）。后一项有代价：检查点列表满了以后也按这个间距删减，对话中途分叉时可能要退回很远（§6.7）。

但存档只覆盖槽位 0。主对话、复盘、Codex 的会话轮流用这一个槽位，不在槽位里的只留在内存的 prompt 缓存里，重启就丢了。10-28 因为测试重启了 12 次，其中 3 次之后的第一个请求把整段对话从头算了一遍：15.5 万、12.2 万和 13.9 万 token，分别用了 93、79 和 81 s。例如 21:30 那次，停机时槽位里是一段 5.5 万 token 的对话，重启后来的却是另一段 13.9

万的。把内存 prompt 缓存里的各段对话也连同检查点存盘、启动后读回可以解决这个问题，但生产上重启不多，我们决定不做。

6.5 对话模板与采样

2026-10-26 接入 Hermes 后，有一次请求的思考越写越长，被客户端以「Repetition Detected」中止，那段思考循环了约 4,700 token。翻看会话记录，思考里「Let me write. / Go.」这样的短句在同一个会话里越来越多：会话的前 2.5 小时一次都没有，之后每条回复里的平均个数从 0.2 涨到 16，最多的一条有 30 个。带工具的对话会保留每一轮的思考，历史里的这类短句一直留在提示词中，模型照着自己之前的写法继续写，越写越多。

我们把 server 用的模板和官方 `encoding/encoding.py` 逐字对照，发现模板沿用了 V4 的写法：开头少了 `<｜System｜>` 和思考强度那一行（`Reasoning Effort: 75 ...`），工具定义和工具结果的位置、拼接方式也不同。按 `encoding.py` 重写后，`template_check.py` 用 10 组对话（system 和 developer 消息、工具定义、多轮工具调用和结果、无工具的多轮思考、几种思考强度）比较 server 的 `/apply-template` 和官方编码，全部逐字一致。对拍中还发现两处：llama.cpp 默认保留历史轮次的思考，官方在没有工具时只保留最后一条用户消息之后的，server 要加 `--no-reasoning-preserve`；llama.cpp 的 Jinja 没有 `from_json`，工具调用的参数要在模板里直接写出。

采样参数原来是 llama.cpp 的默认值（temperature 0.8、top_k 40、min_p 0.05），Hermes 和 Codex 都不在请求里带采样参数，于是改为官方推荐的 temperature 1.0、top_p 0.95，换用 1017 后按其模型卡改为 top_p 1.0（§9）。修复后开的新会话里，这类短句一次也没有再出现；旧会话的历史里已经积了大量短句，要开新会话才能摆脱。

6.6 请求文字里的控制 token

server 原来把整段渲染后的提示词按「识别特殊 token」编码，请求文字里出现的 `<｜User｜>`、`<｜Assistant｜>`、`</think>`、`｜DSML｜` 等字样都会变成真的控制 token，模型会以为对话在正文中间换了轮。我们在生产 server 上确认过；Codex 10-25 有 3 次工具输出含这类字样，智能体读到 llama.cpp 的 `src/llama-vocab.cpp` 时，提示词中间就会多出一个假的结束符。只关掉识别也不够：`<think>`、`</think>`、`｜DSML｜` 在 GGUF 里是用户定义类型的 token，照样会被匹配。

`0018` 按官方 Recipe [8] 的规则编码：只有模板自己写的控制 token 按 ID 编码，其余文字，包括模板里的普通文字，只按基础词表编码。哪些字节来自请求，靠 fork 的 Jinja 引擎给字符串打的输入标记；补丁补上了 `tojson` 和数组 `join` 对标记的传递。这只对 DSV4 的模板生效，续写助手消息的请求走原路径。对拍用 `special_check.py`：§6.5 的 10 组对话，加上 7 组把这类字样放在用户消息、system、工具结果、助手历史、字符串和对象参数、工具定义里的对话，和官方编码器（控制 token 用常量加标记）加去掉 added token 的 HF 分词器逐 token 一致；不含这类字样时和原编码逐 token 相同。40 万字节的提示词编码用 96 ms（原来 104 ms）。输出侧还没有处理：模型自己用普通 token 写出 `</think>` 或工具调用的标签时，解析器仍按文字识别。

6.7 槽位被只共享开头的请求占用

Hermes 每轮结束后会在后台重放整段对话，问模型要不要存技能和记忆（下称复盘）。10-26 晚上一个小时里有三次，主对话的下一条消息要从头重算约 3.6 万 token，每次 20–23 s。原因是三件事叠在一起：复盘请求和主对话只共享前约 4.06 万 token，进来时占了唯一的槽位；server 按共同前缀算保留比例（72%），超过一半就不先把槽位存进内存缓存；而 V4.1 只能退回共同前缀里最新的检查点，当时检查点列表已满，按 `--checkpoint-min-step 32768` 删减后，2 万到 5 万之间一个都不剩，只能退到 20,466。主对话 2 万以后的部分既没有存进缓存，也没有检查点可退。

`0017` 改了两处。一是按检查点算新请求实际能保留多少，比槽位最新的检查点少 2,048 个 token 以上（`ABYSS_SLOT_SAVE_LOST`）就先存进内存缓存，主对话回来时由 §6.2 的逻辑换回。二是把检查点删减的间距和拆分长提示词的步长分开（`ABYSS_CKPT_THIN_STEP=8192`）。`slot-takeover.sh` 复现了这个场景：6.5 万 token 的对话被一个只共享前 3.8 万的请求占用后，下一轮从重算 26,438 个 token（15.0 s）变成 30 个（0.5 s），32 个输出 token 和改动前相同。生产上第一次触发是 Codex 的一个辅助请求，主对话随后只算了 656 个 token。删减间距那一项在这个测试里没有被触发（36 轮还没有填满检查点列表），还要补测。

6.8 智能体客户端的额外请求

除了主对话，Hermes 还会发几类请求，都和主对话抢唯一的槽位：

- **复盘**：见上一节。它可能生成上千 token（一次写技能文件生成了 5,700 token），用户的新消息会让它停下。
- **上下文压缩**：窗口用到约 75% 时，先让模型写摘要，再把摘要和最近的消息从头预填充。有一次摘要生成了 8,189 token、用了 259 s，接着重新预填充 36,711 token、22.7 s。我们让压缩请求不思考（Hermes 的 `auxiliary.compression.reasoning_effort: none`）。另有一次，用户在压缩进行中发来消息，压缩被取消，但请求没有断开，server 仍把 8,490 token 的摘要生成完，用户的消息多等了约 3.5 分钟。
- **命令审批**：默认的 smart 模式让模型审批命中危险规则的终端命令，一天约 23 次，每次的提示词约 800 token、从头算，4.4–4.7 s。改为由用户在 Telegram 上批准（manual）后，这类请求就没有了。

6.9 上一轮回答不再重算

11-07 我们按生产日志统计了每个请求从槽位用上了多少（`prefix_reuse.py`，10-31 到 11-07 的 460 个请求）：429 个后续轮次预填充的 76.8 万 token 里，有 39.1 万（51%）其实已经在槽位里，就是上一轮的回答加 4 个 token。按新提示词和槽位的公共前缀看，311 轮和槽位对到了回答末尾 64 个 token 以内，75 轮断在回答中间，43 轮断在回答开头（模板丢掉了之前轮次的思考，这是官方格式的设计）。原因有两个。

一是检查点的窗口。fork 让 `deepseek4` 的 `llama_model_n_swa` 返回 0：DSV4 的内存只能退回检查点，检查点是一个位置上的完整状态。用同一种内存的 `deepseek41` 却返回滑窗的 128。

server 拿它当余量，新提示词完整包含槽位时，也要去找离末尾 128 个 token 以外最新的检查点，于是退回上一轮提示词的末尾，把回答重算一遍（§6.2 的修复里早就碰到过这条规则，当时没意识到它对 V4.1 是错的）。我们让 V4.1 的检查点窗口也为 0，这样的提示词就从槽位末尾接着算。后来对比上游时发现，上游 10-25 已经做了同样的修正，我们改用上游的写法（`0042`）。

二是回答不总是原样回来。采样出的 token 序列不一定是这段文字的规范切分，客户端把回答带回来后重新分词，公共前缀就断在那里。例如一段回答里模型生成的切分是 READ、ME、.md、edi、ately，同一段文字重新分词却是 READ、ME、.m、ded、i、ately，从第三个 token 起就对不上了；temperature 1.0 比贪心更容易碰到这种情况。`0041`（`ABYSS_GEN_CKPT=256`）在生成时每 256 个 token 存一个检查点：在采样出的 token 进批次之前存，这时内存正好是提示词加已生成的 token，和预填充在每批之前存一样。每个请求最多留 8 个，满了隔一个删一个、间距加倍，让它们分布在整段回答上。每个检查点约 1.5 ms，1,024 token 的回答存 3 个，解码速度测不出差别。

模拟智能体会话（`gen-ckpt.sh`，10 轮、上下文 4.8–6.6 万，后 5 轮接在 1,024 token 的回答后面）：原来每轮重算 1,028 个旧 token，这 5 轮的预填充平均 3.02 s；只改检查点窗口时，9 轮里 7 轮从末尾接着算，2 轮断在回答中间仍要重算；两项都改后，9 轮全部只重算 1 个 token，平均 2.46 s（−19%）。数值上，下一轮看到的上一轮回答的 KV 来自生成时写下的（每轮内部生成时本来就这样用），不再是重算的，对重算的 KL 约 1e-3，和专家延后同一档。上线后的生产（11-07 下午，45 个后续轮次，上下文 10.4–13.1 万）里，30 轮从槽位末尾接着算（预填充平均 1.91 s），4 轮退回回答里的生成检查点，11 轮照旧退回上一轮提示词的末尾；后续轮次重算槽位里已有 token 的比例从 51% 降到 19%。剩下没接上的，7 轮断在回答的第一个 token，是模板按设计丢掉了之前的思考；3 轮断在工具调用的参数里：server 按生成的顺序拼参数、原样保留，模板也按收到的顺序写回，多半是客户端存回对话时改写了参数。

6.10 CUDA graph 空闲后不清

CUDA 后端的图缓存原来每 5 s 扫一遍，删掉 10 s 没用过的图。对话每轮之间（用户在想、工具在跑）常停过 10 s，长的预填充也用不到解码的图，于是下一轮第一次解码时，所有解码图连同正要查的那张一起被删掉，每张都要先直接执行一次、再捕获一次。上游后来改成 10 s 没查过图就给所有图再留 10 s，扫的时候也不删正要查的那张，我们照搬（`0043`）。用小模型在 GPU1 空余的显存上验证（不停生产）：两次请求隔 15 s，旧编译空闲后重新预热了 1 次，新编译 0 次，输出相同；生产每 token 约 46 次图启动，估计每轮开头省 15–30 ms。

6.11 多请求系统的前缀缓存

vLLM [15] 和 SGLang [16] 把 KV 放在共享池里，vLLM 按位置分块（默认每块 16 个 token），SGLang 分页；前缀缓存缓存的就是这些块和页，区别在于怎么认前缀。vLLM 给每个填满的块按「父块的哈希加本块的 token」算哈希，全局一张表从左往右查；SGLang 用基数树，请求结束时把提示词和回答插回树里，只淘汰没被锁住的叶子。两家对 DeepSeek V4 都有专门的做法。vLLM 把压缩 KV 每 256 个原始位置作为一个逻辑块，压缩器的滚动状态当成滑窗 KV 按块存，任何 256 的边界都是现成的恢复点，不用打快照。SGLang 用一棵树索引虚拟的

全 token 槽位，映射到滑窗和两种压缩比的三个池；出了所有请求的滑窗就丢掉滑窗的槽位，压缩 KV 留着给前缀匹配。

这两套解决的主要是多请求并发时的共享和去重，我们是单用户、一个槽位，用不上。能借的是 vLLM 的那个做法：一次调用里每 256 个位置留一份压缩器状态，提示词里任何 256 的边界都能当检查点，不用拆调用（在 CED 下每拆一次就要多读一遍专家），改写较早的消息、复盘这类只共享前一段的请求都用得上。11-08 按 §6.9 上线之后的生产日志看过：后续轮次的公共前缀里白算的只占计算量的 6.5%，大多是模板按设计丢掉的思考；整段重算的长提示词都是新对话，按 256 的边界留状态每次也只能省约 2 s，暂不做。

7 把内存提到 5200、6000 MT/s

4 × 48 GB 双 rank、每通道两条时，AMD 默认只跑 3600 MT/s。内存条标称 5600，SPD 里的 XMP 和 EXPO 配置都是 DDR5-6000、1.40 V。我们在 BIOS 里设 EXPO I，再把频率降到 DDR5-5200，并关掉 Memory Context Restore。stressapptest 用 150 GB 压 1 小时通过，没有硬件错误，内存条最高 62 °C；同一编译重跑的对拍和改之前一致，差别和这个编译两次运行之间一样。

表 16 | 内存从 3600 提到 5200 MT/s 前后（2026-10-25）。

测试	3600 MT/s	5200 MT/s
内存纯读	44 GB/s	54 GB/s
两卡同时从主机读	43.4 GB/s	50.6 GB/s
单卡从主机读	28.9 GB/s	28.9 GB/s
9.6 万上下文多轮会话解码	29.0–31.1 tok/s	30.7–33.0 tok/s
多轮智能体首轮 / 之后每轮	24.0 / 2.9–3.5 s	23.1 / 2.7–3.2 s
新提示词 604 / 2,324 / 4,277 token	2.59 / 3.69 / 4.34 s	2.27 / 3.35 / 4.01 s

各项快了 6–12%，少于我们预计的 15–20%。解码时上传到 GPU1 受 PCIe 限制，每个专家约 0.7 ms，和内存频率无关，只有 CPU 算的那一半变快了。内存带宽也只提高了 23%，理论上是 44%，可能是内存控制器频率被自动设成了内存频率的一半，还没有确认。专家缓存的上传比例（0.5）是在 3600 MT/s 下定的，我们在 5200 MT/s 下重扫了一遍（4.8 万上下文、只追加的会话、每档 6 轮）：0.3、0.4、0.5、0.6 分别是 33.8、33.6、33.3、31.2 tok/s，0.3–0.5 的差别在轮间波动以内，保持 0.5。10-28 专家缓存加到 20.5 GB 后再扫，0.3 仍略好，生产改为 0.3（§4.5）。

11-05 晚我们把频率再提到 DDR5-6000，也就是 SPD 里 EXPO 的配置，其余设置不变（SoC 电压 1.24 V）。stressapptest 同样 150 GB、1 小时通过，它的拷贝带宽 52.0 → 57.9 GB/s，内存条最高 66 °C。表 17 是改前改后。内存带宽这次提高了 11–13%，频率是 15%。两卡同时从主机读只多了 3%，两条 x8 的 PCIe 各约 28.9 GB/s，已经是它们卡住了。生产设置下解码快 1.8%，每 token 只省约 0.3 ms，比我们估的 0.6–0.9 ms 少：有专家延后（§4.8）之后，CPU 算的专家大多已经藏在 GPU 的计算后面，变快的只是没藏住的那部分。预填充的专家经 PCIe 上传，没有变化。11-06 在 6000 下把缓存的几个开关又扫了一遍（每档 24 轮会话）：

上传比例 0.2、0.3、0.4 是 51.74、51.58、51.23 tok/s，热度衰减 0.995（生产 0.99）是 51.03，CPU 线程 24（生产 16）是 51.46，都在 1% 以内，保持原样。

表 17 | 内存从 5200 提到 6000 MT/s 前后（2026-11-05，解码和预填充是当时的生产设置）。

测试	5200 MT/s	6000 MT/s
内存纯读	54–55.5 GB/s	59.8–61.2 GB/s
STREAM Copy / Triad	53.5 / 39.7 GB/s	61.1 / 45.2 GB/s
两卡同时从主机读	53.0 GB/s	54.5 GB/s
9.6 万上下文会话解码	48.32 tok/s	49.19 tok/s
解码统计每 token	19.5 ms	19.2 ms
9.6 万 token 新提示词	39.9 s	39.8 s

这次改 BIOS 还留下一个教训。重新开机后 IOMMU 变成了开着（翻译模式；那次在 BIOS 里还看过 PCIe 的设置，中间意外断过一次电，具体怎么变的不清楚）。两卡之间的直接拷贝（`cudaMemcpyPeer`、nvbandwidth 的所有卡间测试）都返回成功，目标显存却一个字也没写进去，只在内核日志里留下 `AMD-Vi: Event logged [IO_PAGE_FAULT ...]`；CUDA 和 `nvidia-smi topo -p2p` 都不报错，经主机中转的拷贝和单卡显存都正常。生产每个 token 都有跨卡拷贝（层切分处的交接、输出层两半的拼接），这样跑输出会全错，却没有任何报错。我们起初以为是 6000 MT/s 让内存控制器不稳，其实不是：IOMMU 设回 Disabled 后一切正常（卡间单向 27.75、双向合计 104.7 GB/s，校验 0 错误）。所以之后每次改 BIOS、清 CMOS 或断电，起生产前都先跑带数据校验的带宽测试和两卡之间的显存测试，并确认内核日志里没有 `AMD-Vi` 事件。

8 把上下文扩到 19.2 万

智能体会话很容易逼近 12.8 万的上下文。Hermes 在窗口用到约 75% 时压缩上下文：先让模型写一份摘要，再把摘要和最近的消息从头预填充。2026-10-26 的一次压缩用了约 4.7 分钟，其中摘要 8,189 token、259 s，重新预填充 36,711 token、22.7 s。它每轮结束后的后台复盘要重放整段对话，上下文已经长到约 11.8 万 token。Codex 则在 11 万 token 时自动压缩。

显存的限制不在 KV。V4.1 的 KV 是压缩过的，从槽位存档估算每 token 约 5 KB（88,630 token 连同 29 个检查点共 443 MB），25.6 万上下文也只要约 1.2 GB。随上下文变大的是 GPU1 的计算缓冲区，主要是稀疏索引器给压缩位置打分的临时空间，大致和「ubatch × 上下文」成正比：12.8 万上下文时 server 启动预留 4.5 GiB，跑长提示词时涨到 6.1 GiB，GPU1 峰值 96,686 / 97,887 MiB。

我们用 `ctx-size.sh` 逐个试了几种配置（表 18）。每种配置记下加载后的显存，测试全程每 0.5 s 采样一次取峰值；先跑一段 9.6 万 token 的提示词，贪心生成 32 个 token，用来在配置之间对拍；再在开头放一句暗号，灌满语料直到接近上限，最后问暗号，看能否找回，同时测满上下文时的预填充和解码。

有两种腾显存的办法。一是把 ubatch 减半，计算缓冲区约减半，25.6 万上下文的 GPU1 峰值也只有 94,966 MiB；代价是大段预填充慢约 24%（9.6 万 token 从 43.2 s 到 53.4 s），而

表 18 | 各上下文配置的显存与速度（2026-10-27，内存 5200 MT/s）。解码是灌满上下文后生成 256 个 token，开了 `ignore_eos`，答完暗号后的内容容易重复、命中偏高，只适合互相比较。

上下文、ubatch、专家缓存	GPU1 峰值	9.6 万 token	灌满时预填充	解码	暗号
12.8 万、2048、14 GB（原配置）	96,686 MiB	43.2 s	12.8 万，47.9 s	34.1 tok/s	对
12.8 万、1024、14 GB	93,578 MiB	53.4 s	12.8 万，59.4 s	37.7 tok/s	对
19.2 万、1024、14 GB	94,112 MiB	53.1 s	19.3 万，98.4 s	35.0 tok/s	对
25.6 万、1024、14 GB	94,966 MiB	53.1 s	25.9 万，140.0 s	37.7 tok/s	对
19.2 万、2048、10 GB	93,596 MiB	42.9 s	19.3 万，79.7 s	30.9 tok/s	对
19.2 万、2048、12 GB（采用）	**95,604 MiB**	**42.6 s**	**19.3 万，79.4 s**	**33.6 tok/s**	对
19.2 / 25.6 万、2048、14 GB	加载时 GPU1 显存不够（计算缓冲区要 6.7 / 8.6 GiB）				

且智能体每轮只新增 1–3K token，在 ubatch 2048 下多是一个 ubatch，减半后要分成两个，第 22–39 层的专家要从内存多读一遍，这部分没有实测。二是保持 ubatch 2048、缩小专家缓存：12 GB 时 19.2 万上下文的 GPU1 峰值 95,604 MiB，还剩约 2.3 GB，比原配置的余量还大，因为 19.2 万时启动预留的计算缓冲区已经够用，跑长提示词时不再上涨。我们采用第二种；10-28 修掉计算图的浪费、让索引器分块后，专家缓存又加回 16 GB，再借用 GPU0 的空余显存加到 20.5 GB（§4.5）。Hermes 的压缩阈值随之从约 9.8 万推迟到约 14.7 万，Codex 的自动压缩从 11 万调到 16.5 万。

对拍：同一段 9.6 万 token 的提示词，只改上下文大小时输出逐 token 相同（ubatch 1024 下 12.8 万、19.2 万、25.6 万三者 KL 为 0），采用的配置对原配置 KL 5e-6；ubatch 1024 对 2048 是 0.0066，和只换 ubatch 时的差别一致（§5.5）。19.3 万 token 的提示词在 19.2 万和 25.6 万两种配置上的 32 个 token 相同，KL 6e-4。三种长度都能从开头找回暗号。

表 19 | 上线前后的实际解码，取自 server 的解码统计（每 100 个 token 一个窗口）。两段时间的负载不同：前一段是 Hermes，后一段是 Hermes 和 Codex。

	12.8 万、14 GB	19.2 万、12 GB
时段（2026-10-27）	11:50–14:56	15:25–16:28
窗口数	547	617
每 token 中位数	31.1 ms（32.2 tok/s）	32.1 ms（31.2 tok/s）
最慢的 10%	32.8 ms	33.7 ms
专家缓存命中率	72.2%	69.8%
每 token 上传的专家	15.0	约 16
每轮预填充（0.5–3K token）	2–4 s	2–4 s

上线后的结果见表 19。缓存槽位从约 780 个减到约 670 个，命中率降了约 2.5 个百分点，每 token 多上传约 1 个专家，解码慢约 3%；每轮的预填充不变。另一个代价在最坏情况：崩溃后没有存档、或压缩后重建上下文时，19.3 万 token 从头预填充要约 80 s，12.8 万时约 48 s。

9 换用 1017 权重

DeepSeek 在 10 月 17 日发布了 DeepSeek-V4.1-Flash-1017，称为一次小更新：模型卡上 HLE（不用工具）从 36.8 提到 42.6，Terminal-Bench 4.0 从 31.2 提到 42.3，DeepSWE 从 74.2 提到 77.0，完成任务所需的时间和输出长度也更短，推荐 temperature 1.0、top_p 1.0 [5]。DeepSeek 开源的推理框架 DeepInfer [6] 默认用的就是它。1017 的 `config.json`、`tokenizer.json` 和 `encoding/encoding.py` 与原版逐字节相同，96,085 个张量的名字、类型、形状和所在分片也相同，只换了权重。

现成的路都走不通。fork 公开的转换脚本只支持 V4，没有 Engram，导出 DSpark 侧车的选项只认 V4 的架构名；mxxm-t 转换 V4.1 用的脚本没有公开。网上的 1017 GGUF 专家都是 K-quant 或 IQ 量化，不是 MXFP4。NVIDIA 的 NVFP4 版本基于原版，专家按 W4A4 设计，每个权重 4.5 bit（比 MXFP4 大约 6%），只支持 vLLM 和 SGLang [7]。

我们写了一个以现有 GGUF 为模板的转换脚本（`tools/convert/v41convert.py`）：每个输出文件照抄模板的文件头，元数据、张量名、类型、形状和偏移都不变，每个张量的数据按表 20 的规则从 HF 权重转换。Q8_0 和 MXFP4 用 ctypes 调用 fork 编译出的 `ggml_quantize_chunk`，舍入和 fork 完全相同。

表 20 | 1017 各类张量的转换规则，与 mxxm-t 转换原版的做法相同。

HF → GGUF	张量	精度
FP4（每字节两个），每 32 个一个 E8M0 → MXFP4	路由专家，按专家号拼成一个张量	不变，只重排半字节
FP8，32 × 32 块 E8M0 → Q8_0	注意力、共享专家、索引器的 q_b、Engram 的 wkv、草稿模型的 fc	反量化后量化
FP8，每 32 个一个 E8M0 → MXFP4	两张 Engram 表（各 3.84 亿行 × 256）	有损（§2.3）
BF16 → F32	各种 norm	不变
照抄	嵌入、输出、路由门、压缩器、索引器投影（BF16）；sink、超连接系数、路由偏置（F32）	不变

规则先在原版上验证：用 HTTP Range 请求远程读取原版权重，不下载整份，68 种张量各取首、中、尾几块，转换结果和 mxxm-t 的 GGUF 逐字节相同；模板里 1,084 个张量全部找到了来源，没用到的 309 个是视觉部分和多模态用的 `gate.bias_vl`。然后转换 1017：475 GiB 权重下载约 90 分钟，48 个分片的 sha256 都已核对；6 个进程并行，按块写到固定偏移，读写都不进页缓存，免得挤掉 server 正在用的 Engram；可以边下边转，专家等 285.6 GiB 用了 3.8 分钟，两张 Engram 表 97.6 GiB 用了 4.6 分钟。转换后核对了 13 个文件的文件头和大小，并随机抽 400 块重新转换，都与写入的一致。

结果见表 21。1017 的 PPL 低 3.4%；如果转换有错（例如专家排错），PPL 会成倍上升，而不是下降。架构和格式都没变，显存和速度与原版相同。生产在 10-27 切换，采样随之改为 top_p 1.0；旧模型存下的槽位对新权重无效，切换时不读回。1017 的 DSpark 侧车也一起转换好了。

表 21 | 原版与 1017 的对比：同一程序与配置，19.2 万上下文、12 GB 专家缓存。PPL 和 KLD 用 14 段、每段 1,024 token 的语料；这里的 KLD 衡量的是权重的变化，不是数值误差。

	原版	1017
PPL	6.07	5.86（−3.4% ± 0.5%）
对原版的 KLD / top-1 一致率	—	0.075 / 87.7%
GPU1 显存峰值	95,604 MiB	95,604 MiB
9.6 万 token 预填充	42.6 s	42.8 s
19.3 万 token 预填充	79.4 s	79.8 s
满上下文解码*	33.6 tok/s	34.2 tok/s
19.3 万 token 后找回开头的暗号	对	对

* 生成 256 个 token，开了 `ignore_eos`，只适合互相比较（§8）。

10 第 0–19 层的张量并行原型

第 0–21 层的专家都在显存里，解码时两张卡轮流算：GPU0 算第 0–11 层，GPU1 算第 12–21 层，一张卡算的时候另一张在等。让这些层两张卡同时算，就是张量并行。11-07 我们先量了它的前提（`tp-measure.sh`）：每层要做两次 all-reduce（注意力和 FFN 之后各一次），一个 token 的隐状态是 20 KB。内核把部分和直接写进对方的显存、在 GPU 上等对方的事件，录进一张跨两卡的 CUDA graph，每次 4.10 μs，比不交换的对照只多 2.3 μs；用拷贝引擎要 21 μs。每 token 44 次，合计约 0.1 ms。当时按 nsys 的剖析，第 0–21 层每 token 9.28 ms，估计拆开后能省 2.5–2.7 ms（约 10–15%）。

fork 自带整个模型的张量并行（`-sm tensor`）：一个跨两卡的 Meta 设备，注意力按输出组拆，路由专家和共享专家按中间维拆，其余张量两边各放一份，all-reduce 是它自己的 F32 实现，一个 token 的计算录成每卡一张 CUDA graph 回放。但它只能整个模型用，接不上专家放 CPU 和专家缓存。11-09 我们让它和层切分并存（`ABYSS_TP_LAYERS=N`）：设备表在 CUDA0、CUDA1 之后加一个 Meta 设备，前 N 层给它，其余层仍按 `-ts` 切分；`ABYSS_TP_SPLIT` 是 GPU0 的份额，专家按 128 行一档拆。原型能跑通，中间踩过四处：

- Meta 设备只缓存一份子图的拆法，每次调用的子图不同就要重拆（主机每次 2–3 ms），所以并行的那几层中间不能夹着别的段。第 1、14 层的 Engram 在 CPU 上取行，改用 fork 自带的选项在主机上取好、当输入传进来。`engram_q`、`engram_k`（BF16，各 40 KB）原来按乘法测设备支持，CUDA 没有 BF16 的乘法，它们被放在了 CPU（生产里也是这样），每层多出两小段 CPU；改按类型转换测之后（结果逐位相同），每 token 只调一次 Meta 设备、整段回放，主机只用 27 μs。
- 第 20 层的 HC pre 融合，输入全来自第 19 层，被分到了 Meta 设备上，原来的判定因此把所有层的这个融合都关掉了；现在把 Meta 设备也当作同一组卡。
- 视图节点不开新段，第 20 层压缩器状态的视图落进了 Meta 段，按主机上的视图同样处理。
- 第 19 层到第 20 层的交接，原来是主机等 Meta 设备做完、再经主机内存拷，改成在

GPU1 那一路的流上同卡直拷，目标流在 GPU 上等事件。这只让预填充快了（9.6 万 token 提示词 32.5 → 29.1 s），解码时第 20 层本来就要等第 19 层。

显存才是关键。调度器给每个设备一块计算缓冲区、取各段的峰值，Meta 设备因此在每张卡上多一块，1.7–1.85 GB。GPU0 本来就有 4.7 GB 的计算缓冲区，主要是预填充大批次上传第 22–39 层专家的暂存区。前 20 层、GPU0 份额 0.53 时 GPU0 放不下；份额 0.5、第 20–21 层的专家放 GPU0、专家缓存 14 GB 时，两张卡的峰值是 94.0 / 96.4 GB。要回到生产的 21.1 GB 缓存，还差约 7 GB。

表 22 | 第 0–19 层张量并行的原型和生产的层切分（2026-11-09，同一编译、专家缓存都是 14 GB、固定种子）。生产实际用 21.1 GB 的缓存，同样的会话解码统计约 18.5 ms/token。

	层切分	张量并行
9.6 万上下文会话解码	45.50 tok/s	48.66 tok/s
解码统计每 token	20.6 ms	19.4 ms
专家缓存命中率	71.2%	72.6%
9.6 万 token 新提示词	27.0 s	29.1 s
会话首轮预填充	28.3 s	30.6 s
对生产的 top-20 KL	—	8.8e-3

结果见表 22。同样的缓存下解码只快 1.2 ms/token，不到 11-07 估计的一半。按实测反推，生产里第 0–19 层每 token 约 6 ms，张量并行后是 4.7 ms：norm、超连接、路由、索引器和各种小内核两张卡都要各算一遍，省不下来；11-07 的 9.28 ms 是 nsys 按节点跟踪放大了的（§3.3）。可它要少 7 GB 的缓存：生产用 21.1 GB 缓存时每 token 约 18.5 ms，比原型还快。预填充也吃亏，两张卡一起算前 20 层，§5.7 的流水线就用不上了，长提示词慢 8%。数值上，对生产的 KL 在经过编码器层那一档的底线以内。所以我们停在原型，代码留在 fork 的分支上（`experiments/tp-layers.diff`）。

顺带查出两处还没做的：生产里 Engram 的 `engram_q`、`engram_k` 也在 CPU 上，每 token 多 4 小段 CPU；预填充大批次的暂存区改成只放用到的专家，GPU0 约能腾出 3 GB，可以把第 13–21 层的一部分专家挪过去，GPU1 腾出的给缓存，估计每 token 快约 0.5 ms。

11 和上游 llama.cpp 对比

前面每一项都是和改动之前的自己比。这一章换个参照：同一份权重、同样的启动参数，上游 llama.cpp [11] 跑得怎么样；再用三个公开智能体 benchmark 的子集量一遍。

11.1 代码

11-07 晚我们把 fork 和当时的上游（`e95426f16`）按补丁逐个比了一遍。fork 的 server 和 common 停在上游 10-09 的版本，CUDA 部分也早于 10-17 的几处改动，差约一个月。上游 10-25 自己加了 V4.1，是另起的一套实现：张量名和我们的 GGUF 不同，两张 Engram 表合成了一张；它的转换器把 Engram 表和稠密的 FP8 层都写成 BF16，表约 393 GB，是我们

MXFP4 的约 3.8 倍；槽位存档的格式也不同。专家缓存从 fork 分出去以后，上游重写了三遍。表 23 把我们的补丁按上游有没有分成三类。

表 23 | 我们的补丁和上游 `e95426f16` 的对应（2026-11-07）。

类别	补丁
上游已有	稀疏索引器（0006；上游默认开，只有我们对索引的 q、k 做 FP4 舍入）、解码注意力只取 top-k 行（0008）、未命中分给 CPU（0009；上游等缓存满了才分）、两处崩溃修复（0010、0011）、解码的 Engram 预读（0028）、上一轮回答接着算（0042 照上游改）、工具调用解析（0002；上游要模板里写着字面的标签才认 V4.1）
做法不同	大图切段（0038；上游每 512 个节点切一刀）、缓存淘汰（0030；上游按衰减频率淘汰，没有准入门）、Engram 预取（0004）、索引器分块（0020；上游只给不融合的路径分块）、控制 token（0018；上游标了哪些字节来自请求，分词仍识别特殊 token）、调度器的异步输入（0025 的一部分）、超连接融合（0032 的一部分）、矩阵乘的尾部（0036）
只有我们有	CED（0003、0005）、大批次分流和用缓存（0014、0023）、末段一遍算完（0015）、计算图显存（0019）、专家延后（0025）、超连接 K 切分（0027）、KV 拼装融合（0032）、输出层分两卡（0034、0040）、prompt 缓存与槽位（0012、0017）、采样（0029、0035）、生成检查点（0041）、NVMe 保活（0039）、两卡流水线（0044）、拆分和等待的修正（0045、0046）、调度器预留显存的 bug（0013，上游还在）

上游超过 32 个 token 的批次照样把用到的专家全部上传，CUDA 也还不能跳过不算的专家。换到上游，等于在它的 V4.1 实现上重写模型侧、重新转换 GGUF、在改写过三遍的专家缓存上重做缓存补丁，而收益大的部分它都没有，所以我们不换，只拿回有用的修复：n_swa（0042）和空闲后不清 CUDA graph（0043）。11-08 把这段时间上游的其他修复逐个看过，大多用不上：有的只影响带偏移的量化类型，有的只在显卡不支持 VMM 内存池时用到，有的只在投机解码回退时起作用；索引器内核的 bank 冲突修在上游重写过的新内核上，要拿就得把整个内核搬过来、重新对拍，暂不做。反过来能提给上游的有 0013、0027 和 0045 的拆分判断，其中 0013 和上游一个还开着的 PR 里的改动相同。

11.2 同一份权重的实测

为了只比运行时，我们没有用上游的转换器，而是把生产的 GGUF 改成上游读 V4.1 的布局（`to_upstream.py`）：权重逐位相同，只改名字和组织方式。索引器压缩器的 kv 和 norm 改名；`engram_q` 和 `engram_k` 相乘成上游的 `engram_gate`（BF16 相乘，在 F32 里是精确的）；两张 Engram 表首尾相接成一张，Engram 头的偏移改成在合并表里的位置；V4.1 的元数据键换成上游的名字。只有专家的分片直接软链接，其余重写，约 172 GB。哈希用的常数按上游转换器的算法重算，和我们的逐项相同；上游要建的 1,003 个张量都在，抽查的数据和源文件逐字节相同。启动参数和生产相同，另设 `GGML_CUDA_MMQ_PREC=q8`（上游在 Blackwell 上对 MXFP4 也默认 W4A4）；模板里加一行含字面标签的注释，上游才按 V4.1 解析工具调用，两边渲染出的提示词逐 token 相同。上游 GPU1 的计算缓冲区 7.4 GB（我们分块后约 3.2 GB），专家缓存最多给到 16 GB；交给 CPU 的比例扫了 0、0.5、0.7、0.9，取最好的 0.7。

表 24 | 同一份权重、同样的启动参数，生产和上游（2026-11-07，两档前都把模型文件清出页缓存，固定种子）。生产这一列是 0044–0046 之前的，之后长提示词又快了约 35%（9.6 万 token 26.8 s）。

	生产	上游
9.6 万 token 新提示词	41.7 s	220.5 s
4.8 万 token 新提示词	21.0 s	109.4 s
填满 19.3 万 token	83.8 s	462.2 s
智能体之后每轮（1,000–2,300 个新 token）	2.36 s	4.38 s
9.6 万上下文会话解码	49.19 tok/s	39.34 tok/s
智能体的 1,024 token 回答	52.10 tok/s	41.28 tok/s
19.3 万 token 之后解码	49.5 tok/s	41.3 tok/s
专家缓存 / 命中率	21.1 GB / 约 80%	16 GB / 约 72.5%
显存峰值（GPU0 / GPU1）	96,657 / 95,885 MiB	95,435 / 95,597 MiB
PPL，批处理路径	5.8651	5.8394
PPL，逐 token 的解码路径	5.8526	5.8630

结果见表 24。上游的预填充慢 5.3–5.5 倍，原因是没有 CED：每个 2,048 token 的 ubatch 都要把第 22–39 层几乎全部专家（约 130 GB）经 PCIe 传一遍，约 4.5 s，9.6 万 token 是 47 个 ubatch，约 211 s，和实测相符。生产的解码快 20–27%（每 token 20.3 对 25.4 ms）：上游的缓存小、命中率低，也没有专家延后、准入门、K 切分、输出层分两卡和小内核融合。两边都能在 19.3 万 token 处找回开头的暗号，加载都是 48–49 s。

数值上，8 段语料各贪心生成 128 个 token，两边的相同前缀 0–54 个，top-20 KL 平均 1.05e-2、中位数 3.8e-3，分岔处都是近乎平局的候选；9.6 万 token 提示词前 7 个相同，KL 5.0e-3。这在「经过编码器层」那一档底线的上沿，符合两套独立实现的差别。PPL 用同一个程序、14 段各 1,024 token 的语料，算每段的后一半：批处理路径上游低 0.44%（± 0.18%），逐 token 的解码路径生产低 0.18%（± 0.21%），KLD 平均 0.009–0.010，top-1 一致 95.5–95.9%。两边之间的差别和各自两条路径之间的差别同一量级（生产解码路径比批处理低 0.2%，上游高 0.4%），都看不出错误；PPL 只能说明这一点，说明不了哪边更接近官方实现。

11.3 三个公开 benchmark 的子集

自己写的测试难免偏向自己的负载，所以我们又从公开的 benchmark 里挑了三个面向智能体推理的（表 25）。它们原本面向多卡集群和高并发，这里按本机的条件跑：一个槽位、并发 1、上下文 19.6 万 token，从官方数据里截一个子集，客户端和服务器在同一台机器上，每一档都新起服务器。所以跑出的数只能在这两个服务器之间比，都不是官方结果，不能和官方结果比较。其中 MLPerf Agentic Inference 只用了它的数据和客户端：子集、并发和没跑 SWE-bench 那段都不符合 MLPerf 的规则，所以这里的数不是 MLPerf 结果，没有经过 MLCommons 验证（Result not verified by MLCommons Association），下文简称「MLPerf 子集」；AgentX 和 TraceLab 也不是有效提交。MLPerf 是 MLCommons Association 的商标。

我们跑了三次：11-07 在生产和上游上跑 MLPerf 子集和 AgentX；11-09 加了 TraceLab，生产三个都跑，上游只跑 TraceLab（上游的编译和权重没变）；11-10 只跑生产，看 0046 的效

表 25 | 用到的三个公开 benchmark 的数据和客户端。这里跑的都是子集，不是官方结果。

benchmark	负载	这里怎么跑
MLPerf Agentic Inference（MLCommons）[12]	真实文本的多轮轨迹：编程（只用一个 bash 工具修仓库）和客服对话（约 15 个工具）；每轮按记录的历史重拼请求，模型真的在思考和调工具	官方 V4.1 配置的采样参数照用（温度 1.0、top_p 0.95、思考强度 max），截 3 段对话共 54 轮（一段 38 轮的编程、两段 8 轮的客服）
InferenceX v3 AgentX（SemiAnalysis）[13]	393 条真实 Claude Code 会话，只有每次请求的 token 数和 64 token 一块的哈希，客户端按块拼出文本，前缀共享和真实会话一样	官方客户端的 agentx 场景，去掉峰值超过 19 万 token 的会话，900 s 窗口
TraceLab（华盛顿大学）[14]	43 个用户的真实 Claude Code 和 Codex 会话，只有每轮的 token 数、缓存前缀长度和时间	用 AgentX 的客户端按同样规则回放（因此不是有效提交），先去掉峰值超过 19 万的会话树，900 s 窗口

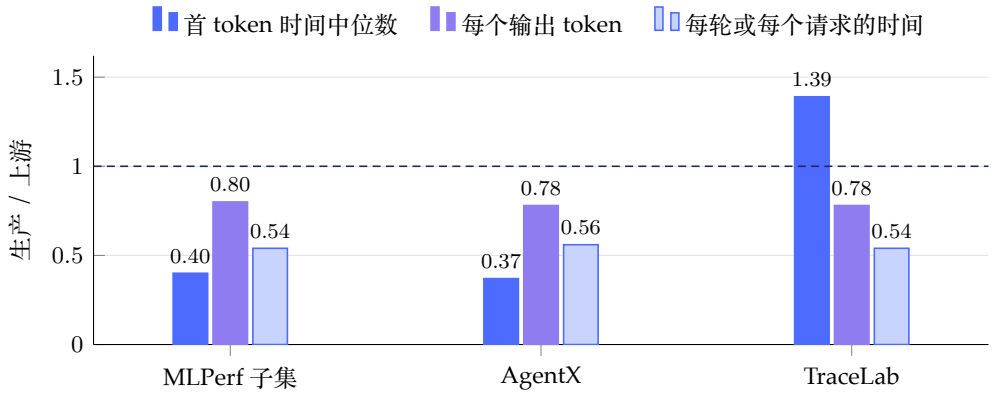


图 10 | 三个 benchmark 里生产对上游的比值，低于 1 是生产更快（虚线是持平）。都是自己跑的子集，不是官方结果；MLPerf 子集不是 MLPerf 结果，未经 MLCommons 验证。MLPerf 子集是 11-07 的 54 轮，每轮时间取平均；AgentX 是 11-07 两边都测到的 22 个请求，TraceLab 是 11-09 的 16 个，时间取合计。

果。AgentX 和 TraceLab 的窗口是固定的，快的一边会测到更多、更后面的请求，整体的百分位数不能直接比，所以只比两边都测到的那些请求（同一个种子，请求和输出长度完全相同）。图 10 是生产对上游的比值。

MLPerf 子集的三次运行见表 26，交互速度是输出 token 总数除以每轮时间之和，已经把输出长度不同算进去。内联准确率只用来确认输出没有出错：这里只有 3 段对话，各次的差别在抽样波动以内，也不能和 MLPerf 规定的准确率门槛比较。AgentX 的 22 个相同请求（提示词平均 9.7 万 token，88.5% 来自缓存）：首 token 时间平均 4.61 对 16.73 s，输出 token 间隔 19.6 对 25.0 ms，请求时间合计 484 对 858 s；预热时冷算 65,756 token 的前缀，生产 30.0 s、上游 166.7 s，有了 0044 之后生产是 20.9 s（11-09）和 20.1 s（11-10）。

TraceLab 有一处是生产输的。16 个相同请求（提示词平均 16.1 万 token）的首 token 时间平均 14.07 对 33.72 s、请求时间合计 440 对 820 s，仍是生产快，冷算 14.4 万 token 的前缀也是生产 45.5 s、上游 358.8 s；但首 token 时间的中位数是 10.98 对 7.92 s，缓存用上的比例

表 26 | MLPerf Agentic Inference 子集的三次运行（3 段对话 54 轮，输入逐轮相同；温度 1.0 采样，输出长度各不相同）。不是 MLPerf 结果，未经 MLCommons 验证（Result not verified by MLCommons Association）。11-07 到 11-09 之间生产上了 0044、0045，11-09 到 11-10 之间上了 0046。

	11-07 上游	11-07 生产	11-09 生产	11-10 生产
首 token 时间中位数 / p90 (s)	3.99 / 15.56	1.60 / 4.68	1.61 / 4.57	1.37 / 4.08
每个输出 token，中位数 (ms)	25.6	20.5	20.4	20.4
每轮延迟，平均 (s)	32.7	17.5	16.2	14.2
每轮输出 token，平均	928	736	690	608
交互速度 (tok/s)	28.42	41.96	42.54	42.94
内联准确率（47 轮）	0.527	0.572	0.591	0.578

76% 对 93%。原因在这份数据合成前缀的方式和 §5.5 的末段合并。它只有 token 数，读取器按 64 token 一块、按记录的缓存前缀长度合成共享部分，新请求只和上一个共享到最后一个整块，都断在上一条用户消息的最后一块里，离上一个提示词末尾 6–57 个 token。末段合并为了少拆一次调用，不在最后 2,700 个 token 里建检查点，最后一条用户消息的开头没有检查点，只能退回很远的那个，每轮重算 1.4–2.3 万 token；上游在最后一条用户消息开头和末尾前 2,052 处都切一刀，每轮只重算几百个。我们的客户端都在末尾追加，从槽位末尾接着算（§6.9），碰不到这种情况；可以做的是一段会话里出现过「新请求断在上一条用户消息里」之后，给这段会话保留那一刀，还没做。

11-10 和 11-09 比，中间只上了 0046：MLPerf 子集的首 token 时间中位数 1.61 → 1.37 s，AgentX 30 个相同请求 3.67 → 3.22 s，TraceLab 33 个相同请求 9.67 → 9.26 s，正是 0046 省下的每轮预填充，输出 token 间隔不变。三个 benchmark 的结论和自己的对比一致：每个输出 token 的时间是上游的 0.78–0.80（约 20 对 25 ms），同样的工作量用时约一半，长提示词的冷预填充快 5–8 倍。

12 结论、局限与未来方向

在一台双卡工作站上，Abyss 以原生 MXFP4 精度、19.2 万上下文运行 DeepSeek-V4.1-Flash（10-27 起为 1017 版）：只追加的多轮会话里单流解码约 52 tok/s（9.6 万上下文，生产上按请求约 50 tok/s）；4.8 万 token 提示词的预填充约 14 s，9.6 万约 27 s，多轮会话每轮约 2 s。同一份权重、同样的启动参数下，11-07 对比时上游 llama.cpp 的解码是 39.3 对 49.2 tok/s，长提示词的预填充慢 5 倍以上；在三个公开智能体 benchmark 的子集上（不是官方结果），同样的工作量我们用时约为上游的一半。收益来自六类工作：顺着模型结构减少计算（CED 感知预填充、稀疏索引器、解码注意力只取 top-k）；按热度安排数据所在的层级（专家缓存、准入和准入门）；让 CPU、DMA 和两张 GPU 同时工作（未命中分流、专家延后、大批次分流和用缓存、两卡流水线）；减少重复的读取和计算（末段一遍算完、上一轮回答不再重算）；去掉不必要的同步和零碎开销（统计不切图、小内核融合、去掉主机的几处等待）；以及把每一块空出来的显存都给专家缓存。每一项都先剖析、后实现，并用分档的数值底线、对官方实现的对拍和解码路径的 PPL 确认正确性。剖析本身也会骗人：同一提示词重复生成和 nsys

按节点跟踪，都曾让我们高估了收益。

局限。 解码的下限由内存带宽决定：第 22–38 层每层 CPU 算未命中的专家都比 GPU1 那一段长，就算把主机内存的流量全藏进 GPU 的计算里，每 token 也还要约 15 ms（§4.15）；预填充的大批次也已接近「每层专家读一遍」的下限。显存几乎用满：GPU0 峰值时空闲只剩约 0.6 GB（显示器因此不能再接在 GPU0 上），GPU1 约 1.4 GB，再加长上下文或开 DSpark 都得先把专家缓存减回去。唯一改变计算的近似是专家延后，解码路径的 PPL 差别在误差以内。GPU1 冷开机时链路偶尔降速（§6.3），改 BIOS 后 IOMMU 曾被打开、让卡间拷贝静默出错（§7），所以开机和改设置之后都要先验证链路和卡间拷贝。server 只有一个槽位，重启时只存它；会改写最后一条用户消息的客户端，会碰上末段合并去掉的那个检查点（§11.3）。Engram 表从官方的 FP8 降到了 MXFP4（相对误差约 12%，§2.3），我们没有直接测过它对任务分数的影响。请求文字里的控制 token 字样在输入侧已按官方规则编码，输出侧还没有处理：模型自己用普通 token 写出 `</think>` 或工具调用的标签时，解析器仍按文字识别。

未来方向。 前两项来自本机的剖析和 benchmark，之后几项来自对 DeepSeek 开源的推理框架 DeepInfer [6] 和 vLLM 的对照：

- **大批次只暂存用到的专家。** GPU0 的计算缓冲区里约 4.7 GB 是预填充大批次上传第 22–39 层专家的暂存区，按整张专家张量留。只放用到的专家，GPU0 约能腾出 3 GB，把第 13–21 层的一部分专家挪过去，GPU1 腾出的给缓存，估计解码每 token 快约 0.5 ms（§10）。
- **给改写消息的会话保留那一刀。** 一段会话里出现过「新请求断在上一条用户消息里」之后，给它保留末段合并去掉的检查点，TraceLab 这类回放的首 token 时间就不会再输给上游（§11.3）。
- **稀疏预填充注意力。** 索引器按查询分块已经做了（§4.5）。DeepInfer 的预填充注意力还按下标表只取 SWA 窗口和 top-512 行；fork 仍为每个用索引器的层生成稠密掩码，让 flash attention 带着掩码扫过全部压缩位置。照此改写能省掉掩码，也许能收回分块让大段预填充慢的 4–8%。这项不引入近似。
- **解码器有界重放。** DeepInfer 在生产上预填充时，解码器层只算整段提示词的最后 140 个 token，SWA 截在这一段之内，是近似。我们的 CED 是精确的级联，实测智能体每轮第 22–38 层每层仍要算约 1,000 个 token（§5.6）。改成有界重放后，要用到的专家和要算的 token 都会少，`0015` 和 `--checkpoint-min-step 32768` 这两个折中也可以去掉；需要先用 KLD 和 PPL 量近似带来的差别。
- **任意位置都能接上。** vLLM 对 DSV4 的做法是一次调用里每 256 个位置留一份压缩器状态，提示词里任何 256 的边界都能当检查点，不用拆调用；DeepInfer 则在任意 128 对齐的位置靠重算最后 128 个 token 补回滑窗状态。两者都能去掉检查点相关的几个折中，但要改 fork 的 DSV4 缓存管理，工作量最大；看过生产日志，眼下每次只能省约 2 s（§6.11）。

- **DSpark 投机解码**。4 张 RTX PRO 6000 上的 vLLM 单流开启后，聊天从 83.6 提到 146.8 tok/s，智能体从 82.2 提到 130.5 tok/s [9]；DeepInfer 按置信度只验证一部分草稿。本机要验证批也能用上单 token 的那些优化、草稿模型只处理提示词末段、腾得出约 10 GB 显存而不砍缓存，三样都成立才值得重做（§4.16）。
- **输出侧的控制 token**。fork 的解析器拿着 token ID 却按文字识别；改用 ID，模型用普通 token 写出的 `</think>` 和工具调用标签就不会被当真。

参考资料

[1] DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the Limits of KV Cache Compression. 2026. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash

[2] mxxm-t. mx-llama.cpp 与 DeepSeek-V4.1-Flash 的 MXFP4 GGUF. https://huggingface.co/mxxm-t/DeepSeek-V4.1-Flash-GGUF

[3] diffbot. DeepSeek-V4.1-Flash-EXL3-3bpw-2x-RTX-PRO-6000. https://huggingface.co/diffbot/DeepSeek-V4.1-Flash-EXL3-3bpw-2x-RTX-PRO-6000

[4] Strata：三层分工的本地 MoE 推理. https://github.com/Niko1221/Strata

[5] DeepSeek-AI. DeepSeek-V4.1-Flash-1017. 2026. https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash-1017

[6] DeepSeek-AI. DeepInfer 及其部署问答（doc/inference_deployment_qa.md）. 2026. https://github.com/deepseek-ai/deepinfer

[7] NVIDIA. DeepSeek-V4.1-Flash-NVFP4. 2026. https://huggingface.co/nvidia/DeepSeek-V4.1-Flash-NVFP4

[8] DeepSeek-AI. DeepSeek Recipe. 2026. https://github.com/deepseek-ai/deepseek-recipe

[9] Infatoshi. DeepSeek-V4.1-Flash-1017-MXFP4-uncensored，模型卡中的 DSpark 消融. 2026. https://huggingface.co/Infatoshi/DeepSeek-V4.1-Flash-1017-MXFP4-uncensored

[10] kvcache-ai. KTransformers：DeepSeek-V4-Flash. 2026. https://github.com/kvcache-ai/ktransformers/blob/main/doc/en/DeepSeek-V4-Flash.md

[11] ggml-org. llama.cpp. https://github.com/ggml-org/llama.cpp

[12] MLCommons. MLPerf Agentic Inference（mlcommons/endpoints，examples/10_Agentic_Inference）. 2026. https://github.com/mlcommons/endpoints

[13] SemiAnalysis. InferenceX v3: AgentX. 2026. https://inferencex.semianalysis.com/blog/agentx-inferencexv3-does-cuda-moat

[14] uw-syfi. TraceLab：真实 Claude Code 和 Codex 会话的轨迹. 2026. https://github.com/uw-syfi/TraceLab

[15] vLLM. DeepSeek V4 in vLLM；Automatic Prefix Caching 设计文档. 2026. https://vllm.ai/blog/deepseek-v4

[16] L. Zheng 等. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104.

A 补丁清单

Abyss 的全部改动以补丁形式保存在仓库的 `mx-llama.cpp/` 目录，基于 mx-llama.cpp 的提交 `c4d12e2`，按序号依次应用。生产用到 `0046`；其中 `0024`、`0031`、`0037` 没有上线，`0033` 的计时默认关，不设开关时都不起作用。第 0–19 层张量并行的原型不在这个序列里，另存为 `experiments/tp-layers.diff`（§10）。

补丁	作用	开关
0001	server 的专家命中统计	`ABYSS_EXPERT_STATS`
0002	V4.1 的工具调用标签（DSML，标签名带前导空格）	—
0003	CED 感知预填充：解码器层只算可见 token 的 FFN	`ABYSS_CED_PREFILL=1`
0004	Engram 整批预取	`ABYSS_ENGRAM_PREFETCH=8`
0005	CED 感知预填充：注意力也只算子集	同 0003
0006	V4.1 稀疏注意力索引器	`ABYSS_DSV41_INDEXER=1`
0007	`llama-abyss-dump`：导出计算图张量用于对拍	—
0008	单 token 解码时注意力只取 top-k 行	`ABYSS_DSV41_DECODE_GATHER=1`
0009	专家缓存未命中按热度分给上传和 CPU	`ABYSS_MOE_CPU_MISS=1` `ABYSS_MOE_ADMIT=0.3`
0010	修 CED 行映射未分配；未命中分流的全零槽位重复	—
0011	修候选块输入未分配	—
0012	prompt 缓存按能直接用上的 token 数挑选	—
0013	修调度器估算显存时清掉手动指定的后端	—
0014	预填充大批次分流到 CPU	`ABYSS_MOE_BIG_SPLIT=1`
0015	提示词末段一遍算完；CED 余量跟 `n_rs_seq`	`ABYSS_CKPT_TAIL_MERGE=2700` `ABYSS_CED_ALIGN=1`
0016	解码统计：每 N 个解码 token 打印一行每 token 耗时、命中率、上传和 CPU 计算的专家数、缺页次数	`ABYSS_DECODE_STATS=100`
0017	槽位被只共享开头的请求占用前先存进内存缓存；检查点删减间距单独设置	`ABYSS_SLOT_SAVE_LOST` `ABYSS_CKPT_THIN_STEP=8192`
0018	对话提示词按官方 Recipe 的规则编码：只有模板写的控制 token 按 ID 编码	`ABYSS_PLAIN_INPUT=1`
0019	计算图显存：全零张量用到时才分配，没有查询行的层不算查询；可记录每次分配	`GGML_ALLOC_TRACE=1`
0020	预填充的索引器打分和注意力按查询分块	`ABYSS_DSV41_IDX_CHUNK=512`
0021	llama-bench 从环境变量读专家缓存大小	`LLAMA_ARG_MOE_CACHE_MIB`
0022	预填充大批次的统计：每批的专家数和其中缓存已有的	`ABYSS_MOE_BIG_STATS=1`
0023	预填充大批次里缓存已有的专家在缓存所在的卡上算	`ABYSS_MOE_BIG_CACHE=1`

补丁	作用	开关
0024	解码时用下一层的路由门预测专家并提前上传（没有上线）	`ABYSS_MOE_PREFETCH=K`
0025	专家延后：CPU 算的专家晚一层加回残差；调度器异步传小输入	`ABYSS_MOE_DEFER=1`
0026	专家统计在每个 ubatch 之后异步拷回选择结果，不再切图	随专家统计
0027	超连接混合系数的 K 切分	`ABYSS_HC_KSPLIT=1`
0028	解码时对要取的 Engram 行提前发 `MADV_WILLNEED`	`ABYSS_ENGRAM_DECODE_PREFETCH=1`
0029	`dist` 采样的 exp 向量化	`ABYSS_DIST_VEC=1`

0030	缓存按衰减热度淘汰、准入门；CPU 算的专家不读零槽位	`ABYSS_MOE_EVICT=cold`
		`ABYSS_MOE_ADMIT_GATE=1`
		`ABYSS_MOE_SKIP_ZERO=1`

0031	上传的专家也延后（没有上线）	`ABYSS_MOE_DEFER_UPLOAD=1`
0032	超连接和单 token 注意力 KV 拼装的小内核融合	`ABYSS_HC_FUSE=1` `ABYSS_GATHER_FUSE=1`
0033	server 每一步的计时（默认关）	`ABYSS_SERVER_STATS=N`
0034	单个输出时输出层两卡各算一半	`ABYSS_HEAD_SPLIT=0.5`
0035	采样快路径：直接在 logits 上抽	`ABYSS_SAMPLE_FAST=1`
0036	矩阵乘向量的尾部每块多算几行；q8_1 缓存的专用池	`ABYSS_MMVQ_TAIL=1`
0037	q8_1 缓存的专用池只放小项（没有上线）	`ABYSS_Q8_1_CACHE_LEG=1`
0038	大 CUDA graph 分两段启动	`ABYSS_GRAPH_CHUNK=128`
0039	解码时 NVMe 保活	`ABYSS_ENGRAM_KEEPALIVE_MS=4`
0040	输出层只分两半加载，不留整张	`ABYSS_HEAD_SPLIT_ONLY=1`
0041	生成时每 256 个 token 存检查点；每个请求从哪接着算的日志	`ABYSS_GEN_CKPT=256`
0042	V4.1 的检查点窗口为 0（照上游）	—
0043	CUDA graph 空闲后不清（照上游）	—
0044	长提示词预填充两卡流水线	`ABYSS_OVERLAP=1`
0045	检查点拆分算上调用开头的检查点；每个请求的调用日志	`ABYSS_CKPT_COUNT_START=1`
0046	拷主机上的权重前不再让主机等	`ABYSS_WEIGHT_NOWAIT=1`

B 复现

生产配置的启动命令如下。需要打过 `0001–0046` 的编译；内存在 BIOS 里设 EXPO I + DDR5-6000，IOMMU 关（§7）。两个 GGML 环境变量是必需的，`CUDA_SCALE_LAUNCH_QUEUES=4x` 配合 `0044`。权重是用 `v41convert.py` 从官方 1017 转换的 GGUF（§9），模板是仓库里的 `deepseek-v4.1.jinja`；仓库的 `restart-server.sh` 还带上了槽位存取、专家统计和解码统计：

```
GGML_CUDA_Q8_1_CACHE=0 GGML_CUDA_MMQ_PREC=q8 GOMP_SPINCOUNT=100000000 \
CUDA_SCALE_LAUNCH_QUEUES=4x \
ABYSS_CED_PREFILL=1 ABYSS_ENGRAM_PREFETCH=8 ABYSS_DSV41_INDEXER=1 \
ABYSS_DSV41_IDX_CHUNK=512 ABYSS_DSV41_DECODE_GATHER=1 \
ABYSS_MOE_CPU_MISS=1 ABYSS_MOE_ADMIT=0.3 ABYSS_MOE_BIG_SPLIT=1 \
ABYSS_MOE_BIG_CACHE=1 ABYSS_PLAIN_INPUT=1 \
```

```
ABYSS_CKPT_TAIL_MERGE=2700 ABYSS_CED_ALIGN=1 ABYSS_CKPT_THIN_STEP=8192 \
ABYSS_MOE_DEFER=1 ABYSS_HC_KSPLIT=1 ABYSS_ENGRAM_DECODE_PREFETCH=1 \
ABYSS_DIST_VEC=1 ABYSS_MOE_EVICT=cold ABYSS_MOE_ADMIT_GATE=1 \
ABYSS_MOE_SKIP_ZERO=1 ABYSS_HC_FUSE=1 ABYSS_GATHER_FUSE=1 \
ABYSS_HEAD_SPLIT=0.5 ABYSS_HEAD_SPLIT_ONLY=1 ABYSS_SAMPLE_FAST=1 \
ABYSS_MMVQ_TAIL=1 ABYSS_GRAPH_CHUNK=128 ABYSS_ENGRAM_KEEPALIVE_MS=4 \
ABYSS_GEN_CKPT=256 ABYSS_OVERLAP=1 ABYSS_CKPT_COUNT_START=1 \
ABYSS_WEIGHT_NOWAIT=1 \
llama-server -m DeepSeek-V4.1-Flash-1017-MXFP4-00001-of-00012.gguf \
  -ngl 99 -sm layer -ts 12/29 -fa 1 -lm dio -c 196608 -b 65536 -ub 2048 \
  -ot "blk\.(2[2-9]|3[0-9])\.ffn_(gate|up|down)_exps\.=CPU" \
  -ot "blk\.12\.ffn_(gate|up)_exps\.=CUDA0" \
  -t 16 -np 1 --moe-cache-mib 21100 --checkpoint-min-step 32768 \
  --jinja --chat-template-file deepseek-v4.1.jinja --no-reasoning-preserve \
  --temp 1.0 --top-p 1.0 --top-k 0 --min-p 0
```

评估脚本在仓库的 `tools/eval/`，复制到机器上运行，运行前要先停 server，见表 27。三个公开 benchmark 的运行脚本在仓库的 `benchmark/`。

转换脚本在 `tools/convert/`：`v41convert.py verify` 用原版权重抽样对比模板，`convert` 转换，`check` 核对输出。

表 27 | 评估脚本。

脚本	用途
10 月	
`kl.sh`	存基准 logits，比较 KLD
`bench.sh`	解码与预填充测速
`prefill-warm.sh`	server 热缓存预填充
`profile-prefill.sh`、`profile-decode.sh`	nsys 剖析预填充、解码
`dump-indexer.sh`、`indexer_oracle.py`	稀疏索引器的单层对拍
`decode-gather.sh`、`decode_attn_oracle.py`	解码注意力的 float64 对拍
`engram-cold.sh`、`engram-share.sh`	Engram 冷读开销与占比
`m2-cpu-miss.sh`	未命中分流的上传比例扫描
`agent-turns.sh`	多轮智能体会话
`decode_modes.sh`	只追加的长会话里测解码，`CAPTURE=1` 剖析
`prefill-sizes.sh`	几种长度新提示词的预填充
`ctx-size.sh`	各上下文配置的显存峰值、长提示词预填充和找回暗号
`slot-takeover.sh`	槽位被只共享开头的请求占用后，主对话还能复用多少
`template_check.py`	对拍对话模板和官方 `encoding.py`
`special_check.py`	逐 token 对拍对话提示词的编码和官方 Recipe 的规则
`align_sweep.py`	调用长度除以 2,048 的各种余数下的边界扫描
`experts5.sh`	每 token 5 个专家对 6 个的 KLD 和解码速度
`dspark.sh`、`dspark_code.py`	DSpark 投机解码的速度、接受率和显存
`alloc-trace.sh`、`alloc_peak.py`	计算缓冲区峰值时有哪些张量
`idx-chunk.sh`	索引器分块的对拍、显存和长提示词
`cache-decode.sh`、`gpu0-experts.sh`	不同专家缓存大小和张量放置下的解码
`admit-sweep.sh`、`threads-sweep.sh`	上传比例、CPU 线程数的扫描
`quick-knobs.sh`	CUDA graph、图优化和分块大小
`profile-prod.sh`、`nsys_layer.py`	用生产设置剖析解码和一轮预填充，打印一层的时间线
`big-coverage.sh`、`big-cache.sh`	大批次的缓存覆盖率；`0023` 开关前后的对比
`std-bench.sh`	llama-bench 和 AIPerf 的标准口径
11 月	
`cache-gate.sh`	在生产设置上按开关对比：数值、会话测速、缓存统计
`defer.sh`、`greedy_kl.py`	专家延后的对比；多段语料贪心生成、按相同前缀比 top-20 KL
`defer-ppl.sh`、`nll_compare.py`	强制解码，逐 token 配对比较解码路径的 PPL
`prod-decode-profile.sh`	按生产设置剖析解码：逐层、逐内核、CUDA graph 的启动
`expert-trace.sh`、`cache_policy.py`	导出解码的路由；离线比较跨层预测和缓存策略
`engram-decode.sh`、`engram_reads.c`	解码时 Engram 读盘的剖析；冷行的读法和空闲后首读
`host-profile.sh`	解码中主线程离开 CPU 的调用栈和时长
`vram-peak.sh`	加载后和几段负载之后两张卡的显存峰值
`sample-fast.sh`、`seeded_tokens.py`	固定种子逐 token 对拍采样
`gen-ckpt.sh`、`prefix_reuse.py`	上一轮回答接着算的对比；从 server 日志统计槽位复用
`prefill-pipe.sh`、`big-pipe.sh`	长提示词和智能体一轮预填充的剖析，按两张卡分解
`upstream-bench.sh`、`upstream-ppl.sh`	一次停机里对比上游和生产；两条路径的 PPL
`tp-measure.sh`、`tp-proto.sh`	两卡 all-reduce 的延迟；张量并行原型的对比
`gpu-bw.sh`、`gpu_memtest.cu`	开机或改 BIOS 后查链路、带宽和卡间拷贝
`precision_audit.py`	GGUF 各类张量和官方 checkpoint 的逐类核对
`benchmark/run.sh`、`compare-runs.sh`	三个公开 benchmark 的子集；两次运行的对比

C 术语表

按主题分组；同一个词在不同语境下含义不同的（如「解码器层」和「解码」、server 的槽位和专家缓存的槽位），在条目里分别说明。

模型结构

MoE（混合专家） 每层有许多个前馈网络（专家），每个 token 只由路由器挑出其中几个来算。V4.1-Flash 每层 384 个路由专家，每个 token 选 6 个，另有 1 个共享专家每个 token 都算。

路由器（gate） 每层给 384 个专家打分并选出 top-6 的小矩阵；它选中的专家号决定了这一步要读哪些专家权重。

CED（编码器-解码器结构） V4.1 把 40 层分成前 20 层因果编码器（第 0–19 层）和后 20 层解码器（第 20–39 层）。第 20 层用编码器的输出生成所有解码器层共享的全局 KV；解码器层自己的隐状态只能通过 128 个 token 的滑动窗口被其他 token 看到。所以提示词里较早的 token 不必算完解码器层，这是 CED 感知预填充的依据。

编码器层、解码器层 分别指第 0–19 层和第 20–39 层，是模型结构上的划分；和下面推理过程里的「解码」（逐个生成 token）不是一回事。

滑动窗口注意力（SWA） 每个 token 只看自己和前面 127 个位置的 KV（共 128 个）。

压缩注意力、压缩比 把每 2 个或每 1 个 token 的 KV 压成一行，长程注意力只看压缩后的行。第 0–1 层只有滑动窗口，第 2–19 层压缩比为 2，第 20–39 层为 1。压缩 KV 只在第 2、8、14、20 层（KV 源层）写入，其后的层复用。

全局 KV 第 20 层的压缩器生成、被全部解码器层共享的压缩 KV。

稀疏索引器（lightning indexer） 用 32 个头、128 维、FP4 的查询和键给所有压缩位置打分，为每个查询挑出 top-512 个位置；注意力只看这些位置和滑动窗口，共约 640 行。第 2、8、14、20、24、28、32、36 层是索引源层，其余层沿用最近索引源的结果。

候选块 第 20 层按每 8 个压缩位置一块挑出 2,048 块，第 24–36 层只在这些块里重新选 top-512。

超连接（hc） V4 系列对残差连接的扩展：隐状态保留 4 路，每层注意力和前馈网络的前后按学到的系数把它们混合、写回。解码时它带来不少小内核。

Engram V4.1 的查表模块：第 1、14 层按最近 2–4 个 token 的 n-gram 哈希，从两张各约 3.84 亿行的大表里取行。表很大（官方 FP8 约 189 GiB），但每个 token 只读几十行，所以可以放在 NVMe 上。

DSpark V4.1 自带的约 14B 草稿模型，用于投机解码：3 层，每层 128 个专家选 3 个，一次出 5 个 token，读主模型第 37–39 层的隐状态，并用置信度头预测每个草稿 token 被接受的概率。本机试过，暂不采用（§4.16）。

数值格式与量化

MXFP4 4 bit 浮点（E2M1），每 32 个元素共享一个 8 bit 的 2 的幂缩放（E8M0），平均 4.25 bit/权重。V4.1 路由专家的原生格式。

FP8、BF16、F16、F32 8、16、16、32 位浮点。官方 checkpoint 的稠密权重和 Engram 表是 FP8；KV cache 用 F16。

Q8_0 llama.cpp 的 8 bit 整数格式，每 32 个元素一个 F16 缩放，平均 8.5 bit。GGUF 里的稠密权重是这种格式。

W4A8、W4A4 矩阵乘的精度：权重 4 bit，激活量化到 8 bit 或 4 bit。官方参考实现是 W4A8；fork 在 Blackwell 上的预填充默认是 W4A4，我们用 `GGML_CUDA_MMQ_PREC=q8` 改回 8 bit。

bpw 每个权重平均占的位数。方案 A 把专家从约 4.25 bpw 压到 3.0 bpw。

EXL3 ExLlamaV3 的量化格式，方案 A 的公开配方用它。

GGUF llama.cpp 的模型文件格式；本报告起初用 mxxm-t 转换的 MXFP4 GGUF，10-27 起用我们按它的格式自己转换的 1017 版，都是 12 个分片、383 GiB。

推理过程

预填充（prefill） 一次处理整段提示词，写入 KV 并算出第一个 token 的分布。很多 token 一起算，主要受算力和读权重的总量限制。

解码（decode） 每次生成一个 token。每一步都要把用到的权重读一遍，主要受带宽限制。

tok/s 每秒 token 数：预填充指每秒处理的提示词 token，解码指每秒生成的 token。

KV cache 已处理位置的键和值，后面的 token 做注意力时读取。V4.1 的 KV 很小：每个位置只有 1 个 512 维的头，又经过压缩。

llama_decode 调用、ubatch server 把要处理的 token 组成一次 `llama_decode` 调用（上限由 `-b` 决定，本报告用 65536）；一次调用再切成最多 2,048 个 token（`-ub`）的 ubatch，每个 ubatch 把计算图从头到尾跑一遍。

检查点（checkpoint） DeepSeek V4 的缓存不能回退到中间位置。server 在一些位置保存那一刻的完整状态（滑动窗口和压缩器状态），之后的请求如果在这之后才和缓存分叉，就从检查点恢复，不必从头算。

prompt 缓存 server 把处理过的提示词状态（连同检查点）留在内存里，下一个请求前缀相同时直接复用。

server 的槽位（slot） server 同时服务的会话数（`-np`），本报告用 1 个；和专家缓存的槽位无关。

回退环（n_rs_seq） 为投机解码准备的少量回退能力，可以撤销最近几个 token；没开投机解码时为 0，缓存只能退回检查点。

投机解码 用小模型先猜几个 token，再让大模型一次验证，猜对的部分直接接受。

对话模板（chat template） 把多轮消息、工具定义和工具结果拼成模型看到的 token 序列的规则。V4.1 的官方规则在 `encoding/encoding.py`，server 用 Jinja 模板实现，必须逐字一致（§6.5）。

控制 token 对话模板里标记角色和结构的特殊 token，如 `<｜User｜>`、`<｜Assistant｜>`、`</think>`。请求文字里出现同样的字样时，要按普通文字编码（§6.6）。

temperature、top_p 采样参数：temperature 调整概率分布的尖锐程度，top_p 只在累计概率达到 p 的候选里抽取。1017 推荐两者都取 1.0，即按模型原本的分布采样。

有界重放（bounded replay） DeepInfer 的做法：需要重建滑动窗口的状态时，只重算最后 128 个 token，注意力窗口也截在这一段之内，得到近似的 SWA KV；解码器层的预填充也只算末尾 140 个 token。

temperature 0（贪心） 每步都取概率最大的 token，同样的输入一定得到同样的输出，适合对拍。

Abyss 调度器

方案 A、方案 B 方案 A 把专家量化到 3.0 bpw，用 vLLM 双卡张量并行（TP2），低于原生 4 bit，11-04 起不再考虑；方案 B 保留原生 MXFP4，用 mx-llama.cpp 按层切分，是本报告的主线。

张量并行、Meta 设备、all-reduce 张量并行把一层的权重拆到两张卡上同时算，每层算完把两边的部分和加起来（all-reduce）。fork 用一个跨两卡的 Meta 设备实现它；我们让前 20 层用它、其余层照旧按层切分，做到了原型（§10）。

层切分 `-sm layer -ts 12/29`：GPU0 放第 0–11 层整层，GPU1 放第 12–39 层的稠密部分和第 12–21 层的专家，第 22–39 层的专家放主机内存。

锁页内存（CUDA_Host） 不会被操作系统换出、GPU 可以直接用 DMA 读取的主机内存；第 22–39 层的专家（121 GiB）放在这里。

专家缓存、bank、槽位 GPU1 上划出一块显存（bank），分成槽位，每个放一个专家：12.8 万上下文时 14 GB、约 780 个，19.2 万时先是 12 GB、约 670 个，10-28 起 20.5 GB、约 1,140 个，11-07 起 21.1 GB、约 1,170 个；每层有一张从专家号到槽位号的映射表。

命中、未命中、命中率 token 选中的专家已在缓存里叫命中，否则叫未命中；命中率按「token × 专家」计。实际会话里约 65–81%。

LRU 最近最少使用：缓存满了先换出最久没用过的专家。

准入、热度 未命中的专家不是都放进缓存：按衰减热度（每层每步乘 0.99）挑出一定比例（10-28 起为 0.3）上传进缓存，其余由 CPU 算。

准入门、冷淘汰（0030） 只有比要挤掉的那个更热的未命中才放进缓存；要挤掉的是衰减热度最低的槽位，不再按最久没用。

专家延后（0025） 解码时第 22–38 层交给 CPU 算的专家不并进本层的输出，在下一层注意力之前按本层的系数加进残差。残差本身是精确的，近似只在下一层的注意力没看到这部分；好处是 CPU 算这些专家时 GPU1 可以接着算下一层的注意力。

两卡流水线（0044） 长提示词预填充时，GPU0 算下一个 ubatch 的第 0–11 层，GPU1 同时算这一个的后面各层。

生成检查点（0041） 回答生成时每 256 个 token 存的检查点；上一轮回答重新分词后对不上时，退回这里而不是上一轮提示词的末尾。

未命中分流（0009） 解码时把未命中的专家分成两路同时处理：一部分上传到 GPU1 的缓存里算，其余由 CPU 直接用主机内存里的权重算，每个专家只从内存读一遍。

大批次分流（0014） 预填充时超过 32 个 token 的批次，把分到 token 少的专家交给 CPU，其余照常上传到 GPU0。

CED 感知预填充（0003、0005） 解码器层只对后面的 token 还能通过滑动窗口看到的那些 token 计算前馈网络和注意力，KV 照常全部写入。

大批次用缓存（0023） 预填充的大批次里，缓存已有的专家直接在 GPU1 上用槽位算，其余照旧分给 CPU 和上传到 GPU0，三路同时进行。

索引器分块（0020） 预填充时稀疏索引器的打分和注意力每次只算一段查询（生产用 512 个），计算缓冲区不再随「ubatch × 上下文」整块变大。

末段一遍算完（0015） 让提示词最后一段不再被 server 拆成几次调用，并让一次调用的最后一个 ubatch 是满的，解码器层的专家尽量只读一遍。

计算图、split ggml 把一次前向计算表示成计算图，再按算子所在的设备切成几段（split），每段在一个后端（CPU、GPU0 或 GPU1）上执行，段与段之间拷贝数据、同步。

MMQ、MMVQ CUDA 上的量化矩阵乘内核：MMQ 用于超过 8 个 token 的批次（预填充），MMVQ 用于 1–8 个 token（解码）。

CUDA graph 把一串内核录成一次提交，减少逐个启动的开销。节点多的大图在第 128 个节点之后切成两段，第一段跑的时候发第二段（0038）。

flash attention 不显式存出完整注意力矩阵、分块计算注意力的内核。

评估方法

对拍 同一段输入下，把改动后的输出和改动前或参照实现逐项比较。

KLD（KL 散度） 两个配置在每个位置上输出的概率分布之差，再取平均；0 表示完全相同。本报告用 llama.cpp 的 `llama-perplexity` 计算。

数值底线 不改变语义的改动（换内核、换 ubatch 大小、换计算设备）也会带来的 KLD。只经过解码器层时约 0.0005，经过编码器层时 0.005–0.01（§3.1）。

top-1 一致率 两个配置下概率最大的 token 相同的位置所占的比例。

top-20 KL 只用 server 返回的前 20 个候选的概率估算的 KL，用来在单 token 的解码路径上对拍。

PPL（困惑度） 模型对一段文本的平均「意外程度」，越低越好。

参照实现（oracle） 我们照官方 `inference/model.py` 写的 PyTorch 实现，用同样的输入逐层重算，检查 fork 的每一步。

nsys NVIDIA Nsight Systems，记录 GPU 内核、拷贝和 CUDA 调用的时间线，用来剖析每个 token 的时间花在哪。按节点跟踪 CUDA graph 能看到每个内核，但会把图的启动放大约 3 倍；看启动和空闲要按整张图跟踪（§3.3）。

解码统计（0016） server 日志里每 100 个解码 token 一行：每 token 的耗时、专家缓存命中率、每 token 命中、上传和交给 CPU 的专家数、磁盘缺页次数。它不含 `llama_decode` 返回之后的收尾和采样，所以比按请求算的每 token 时间少约 1 ms。

强制解码的 PPL 提示词之后逐个喂参考文本，记下解码路径上每个 token 的负对数似然，两个配置配对比较；比批处理的 PPL 更能反映单 token 解码的改动（§3.4）。

固定种子 温度 1.0 采样时固定随机种子，两档配置在同样的输入下生成同样的会话，缓存的命中率不再随采样起伏（§3.5）。

MLPerf Agentic、AgentX、TraceLab 三个公开的智能体推理 benchmark，分别来自 MLCommons、SemiAnalysis 和华盛顿大学；本报告在单槽位、并发 1 下跑它们的子集，只用来比较生产和上游，都不是官方结果；MLPerf 那一项不是 MLPerf 结果，未经 MLCommons 验证（§11.3）。

llama-bench、AIPerf llama.cpp 自带的基准程序（pp 测预填充、tg 测生成）和 NVIDIA 的推理服务压测工具（测首 token 时间和每 token 时间），本报告用来给出标准口径（§3.6）。

stressapptest Google 的内存压力测试工具，反复读写大块内存并校验，用来确认内存提频后没有错误。

硬件与系统

PCIe 5.0 x8、Gen3/4/5 显卡和主机之间的链路：第几代决定每条通道的速率，x8 是 8 条通道。Gen5 x8 实测约 28.9 GB/s；开机时链路训练失败会降到 Gen4 或 Gen3。

P2P 两张显卡之间直接拷贝，本机经过 CPU 转发，单向约 27.7 GB/s。

MT/s、双通道 内存每秒传输次数和通道数，两者决定理论带宽：双通道 3600 MT/s 约 57.6 GB/s，5200 MT/s 约 83.2 GB/s，6000 MT/s 约 96 GB/s。

IOMMU 让设备的 DMA 地址经过一层翻译的硬件。本机必须在 BIOS 里关掉：开着时两卡之间的直接拷贝返回成功，数据却没有写进去（§7）。

APST NVMe 的自动省电：盘空闲一段时间后进入更省电、但唤醒要几毫秒的状态。解码时偶尔读一次 Engram 冷行，正好撞上唤醒，所以系统盘关掉了它（§4.11）。

2DPC、双 rank 每个内存通道插两条内存；每条内存有两组可以分别寻址的芯片。两者都会让内存控制器难以跑高频率。

EXPO、XMP 存在内存条 SPD 芯片里的超频配置（频率、时序、电压），BIOS 可以一键载入；EXPO 是 AMD 的，XMP 是 Intel 的。

UCLK 内存控制器的时钟。和内存时钟 1:1 同频时延迟最低、带宽最好；1:2 时控制器只跑内存时钟的一半，延迟更高，实际带宽也会打折扣。

页缓存、mmap、MADV_WILLNEED Linux 把读过的文件内容留在内存里（页缓存）；mmap 把文件映射到内存地址，访问时按需读入；`MADV_WILLNEED` 提前告诉内核哪些页马上要用，让它预读。

Max-Q RTX PRO 6000 的 300 W 版本，功耗上限低于 600 W 的工作站版。

Codex、MCP Codex 是 OpenAI 的编程智能体命令行工具，本报告用它作为实际负载的客户端；MCP（Model Context Protocol）是给智能体接入外部工具的协议。